

Backpacks and Black Boxes: Reproducible Science When AI Writes the Code

Tanu Malik

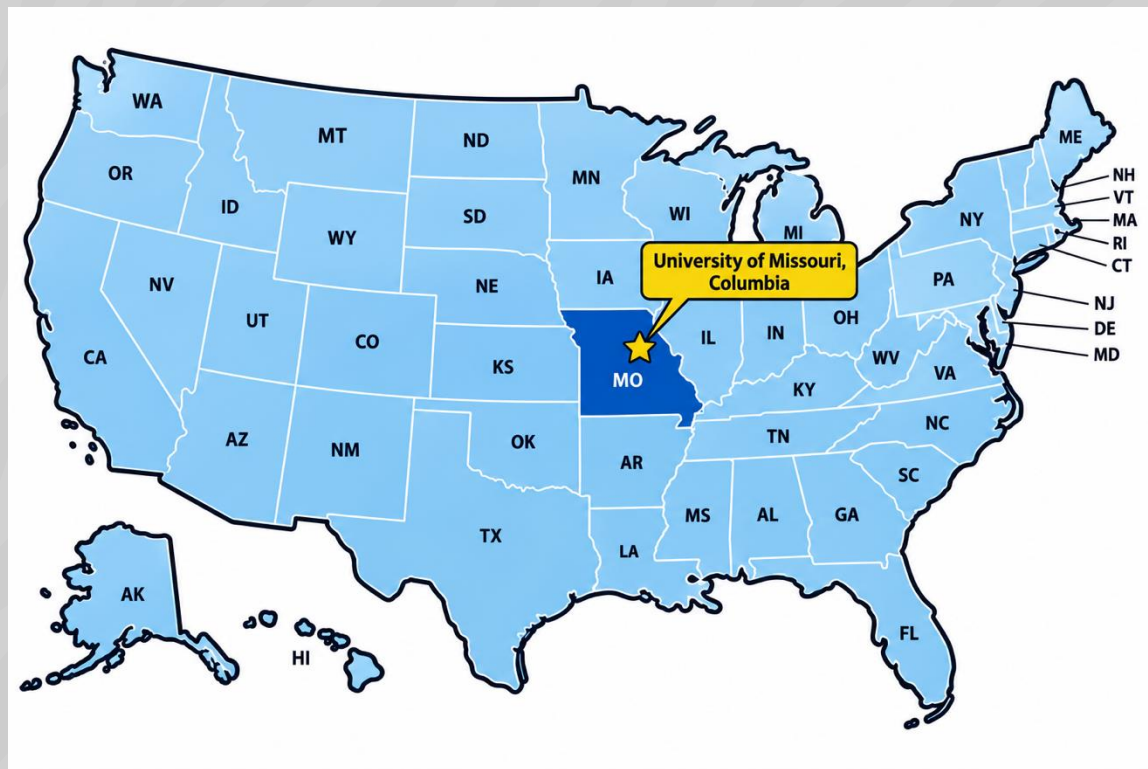
Associate Professor, David E. Payne Electrical Engg. and Computer Science

Director, The RADIANT Lab

University of Missouri

tanu@missouri.edu

1 hour direct flight from Chicago
1.5 hours drive from St. Louis



CoastRunners

- A game on which Open AI trained AI agents
- The goal of the game: win a boat race
 - Player earns higher scores **by hitting targets laid out along the route.**

CoastRunners



Video Source: Faulty Reward Functions in the wild; <https://openai.com/index/faulty-reward-functions>

Is this gaming agent winning the race or failing?

Video Source: Faulty Reward Functions in the wild; <https://openai.com/index/faulty-reward-functions>

Is this agent succeeding or failing?

Specification Gaming: The gap between what we asked and what we got

Amodei, D., et. al. (2016). “Concrete Problems in AI Safety.” arXiv:1606.06565

Krakovna, V., et. al. 2020). “Specification gaming: the flip side of AI ingenuity.” Google DeepMind.

If an AI coding agent gives you a `complete` software artifact, can a human actually run it?

Vangala, B.P, Adibifar, A., Gehani, A., Malik, T., (2026). [“AI-generated code is not reproducible \(yet\): an empirical study of dependency gaps in LLM-based coding agents”](#), In *Workshop on Reproducible AI, AAAI*.

Vangala, B.P, Gehani, A., Malik, T., (2026) [AI-Generated Code Is Not Reproducible \(Yet\): An Empirical Study of Execution Reliability in LLM-Based Coding Agents](#)” ACM REP 2026

300 artifacts → 100 prompts → 3 AI coding agents → Clean environment
Zero manual intervention

Vangala, B.P, Adibifar, A., Gehani, A., Malik, T., (2026). [“AI-generated code is not reproducible \(yet\): an empirical study of dependency gaps in LLM-based coding agents”](#), In *Workshop on Reproducible AI, AAAI*.

Vangala, B.P, Gehani, A., Malik, T., (2026) [AI-Generated Code Is Not Reproducible \(Yet\): An Empirical Study of Execution Reliability in LLM-Based Coding Agents](#)” ACM REP 2026

68.3%

executed successfully out-of-the-box

Vangala, B.P, Adibifar, A., Gehani, A., Malik, T., (2026). [“AI-generated code is not reproducible \(yet\): an empirical study of dependency gaps in LLM-based coding agents”](#), In *Workshop on Reproducible AI, AAAI*.

Vangala, B.P, Gehani, A., Malik, T., (2026) [AI-Generated Code Is Not Reproducible \(Yet\): An Empirical Study of Execution Reliability in LLM-Based Coding Agents](#)” ACM REP 2026

Overall reproducibility

*Experiment conducted
in December 2025*

100 prompts, 3 agents, 3 languages, 300 artifacts

By agent

Agent	Total	Success	Failed (partial)	Rate
Claude	100	73	27 (4)	73.0%
Gemini	100	72	28 (7)	72.0%
Codex	100	60	40 (3)	60.0%
Overall	300	205	95 (14)	68.3%

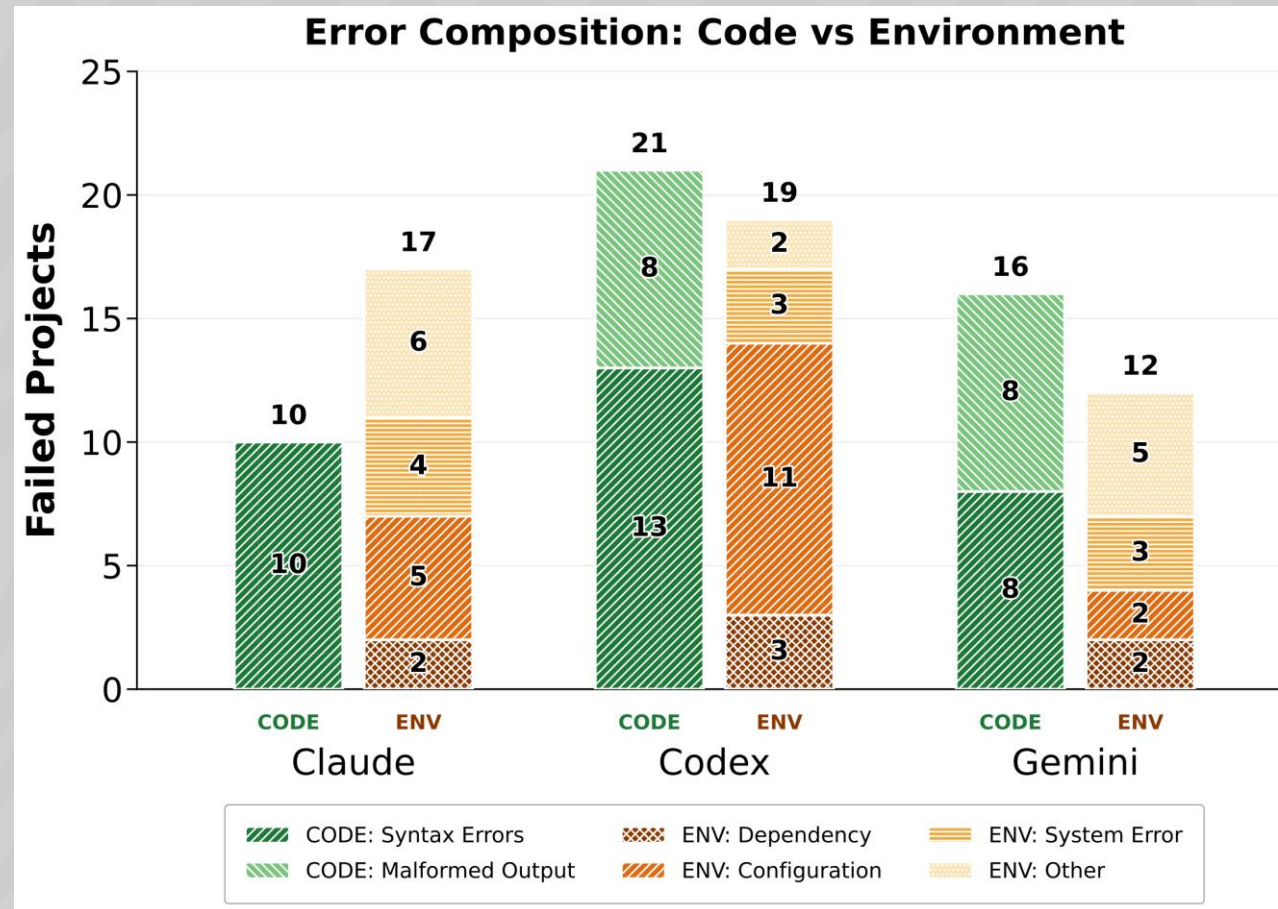
By language

Language	Total	Success	Rate	95% CI
Python	120	107	89.2%	[82.4, 93.6]
JavaScript	105	65	61.9%	[52.3, 70.6]
Java	75	33	44.0%	[33.2, 55.3]

Partial executions are included in the failure count.

205 of 300 artifacts (68.3%) execute out-of-the-box.
The remaining 95 artifacts (31.7%) do not.

Why artifacts fail: code and environment errors



95 failures: 49.5% code, 50.5% environment

If an AI coding agent gives you a complete software artifact, can a human actually run it?

Sometimes—but not reliably (nearly 1 in 3 failed)

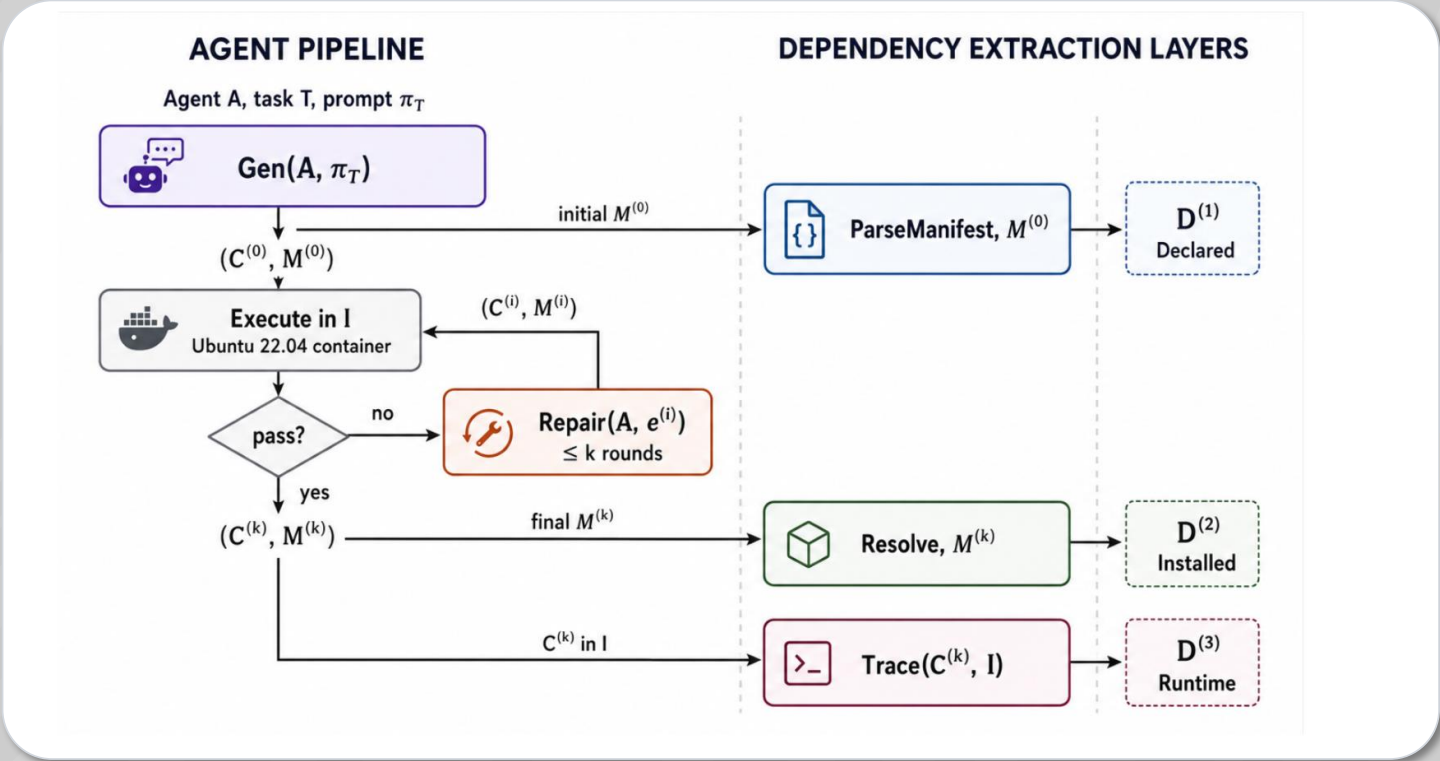
Vangala, B.P, Malik, T., (2026). “Code That Works, Environments That Don’t: Measuring Environment Reproducibility in AI-Generated Software”
Forthcoming in AAAI Magazine.

If an AI coding agent tells you what its software needs, how accurate is that specification?

Vangala, B.P, Malik, T., (2026). “Code That Works, Environments That Don’t: Measuring Environment Reproducibility in AI-Generated Software”, *[Forthcoming in AAAI Magazine. Special Issue Reproducibility in AI](#)*

Environment Reproducibility: Protocol

Generation, repair, and independent dependency measurement

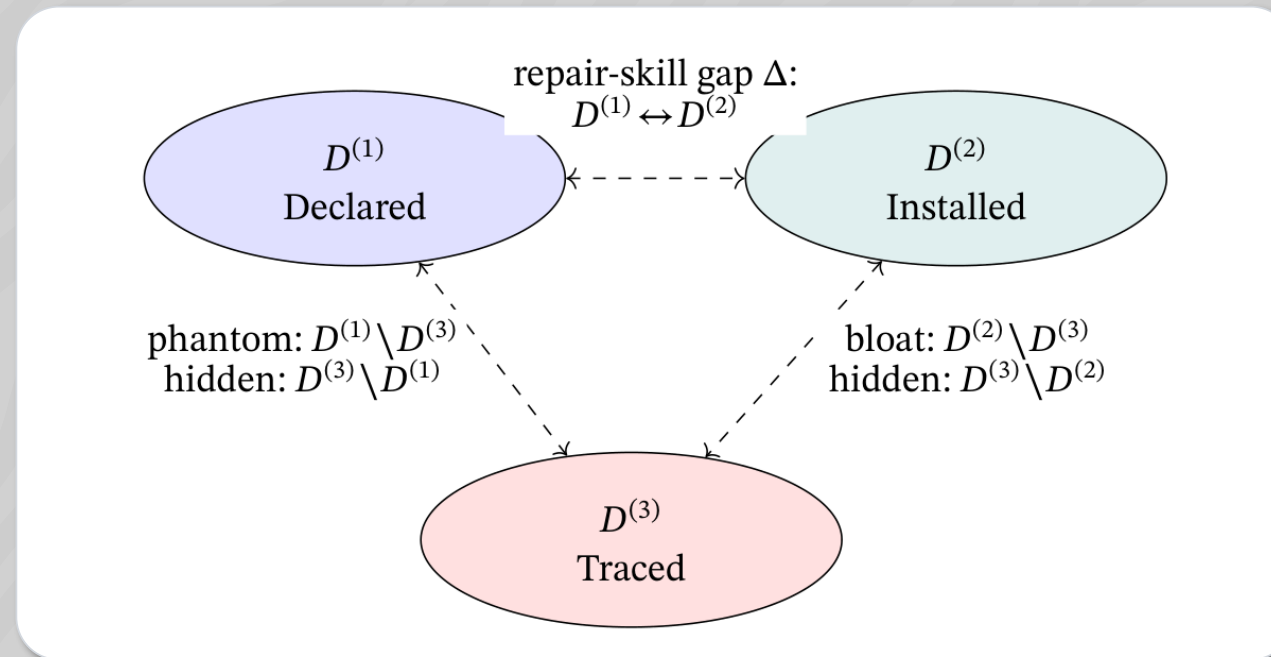


Dimension	Study design
Tasks	50; eight domains
Ecosystems	Python, Java, JS, C++
Coding Agents	Claude, Gemini, Codex
Primary instances	$50 \times 4 \times 3 = 600$
Repair budget	Up to 10 cycles
Extra Claude trials	$50 \times 4 \times 2 = 400$

Fresh Ubuntu 22.04 containers; explicit prompts; baseline toolchains recorded per ecosystem.

Agent Environment Specification

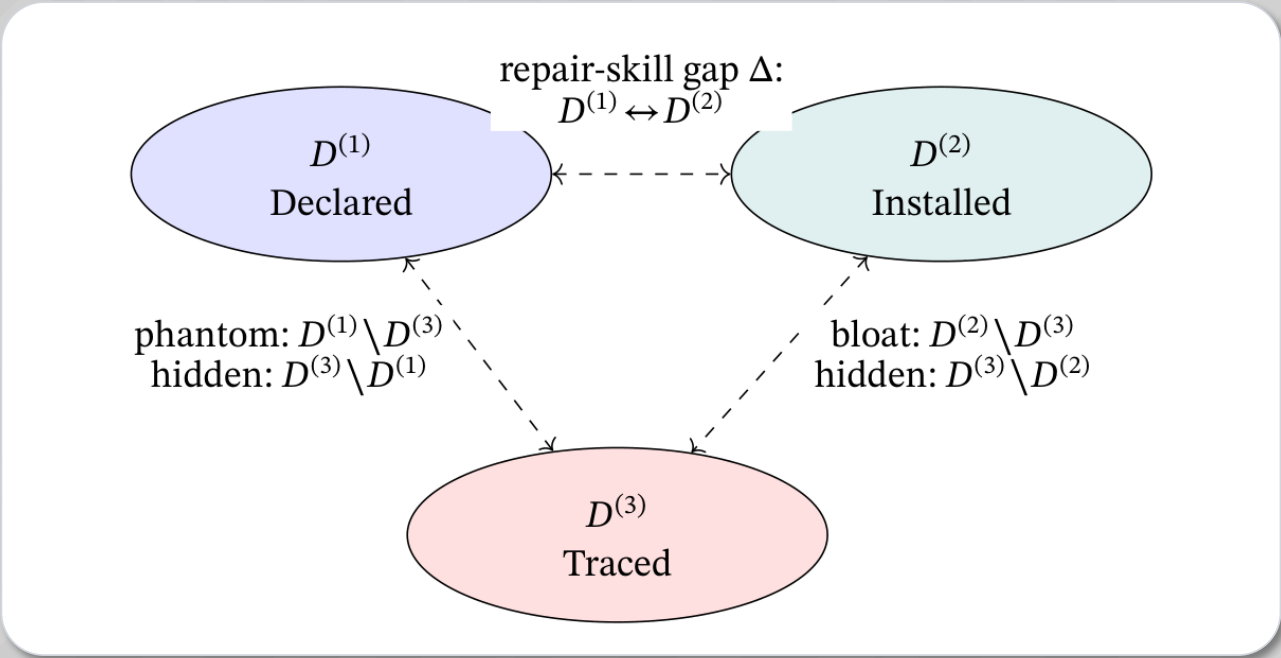
Declared, installed, and traced dependency layers



declared: what the agent initially says the artifact needs
installed: the final environment after installation and repair.
traced: what we actually observe during a successful execution.”

Measure Divergences in Agent Environment Specification

Declared, installed, and traced dependency layers

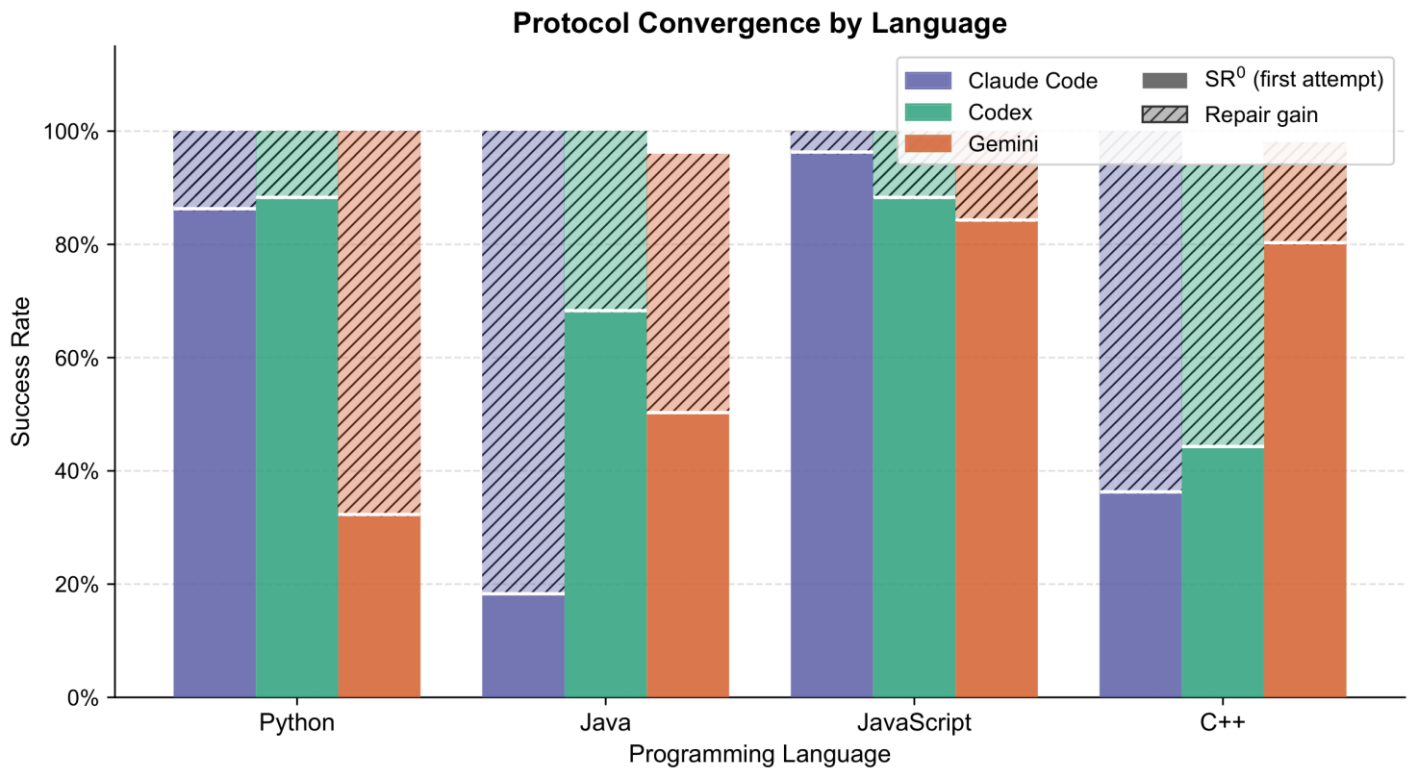


$D^{(1)}$: the initial manifest.
 $D^{(2)}$: the final installed environment.
 $D^{(3)}$: the traced successful execution.

Set differences reveal:
phantom aka agent hallucination,
hidden aka under specification by agent,
and **bloat** aka overspecification by agent

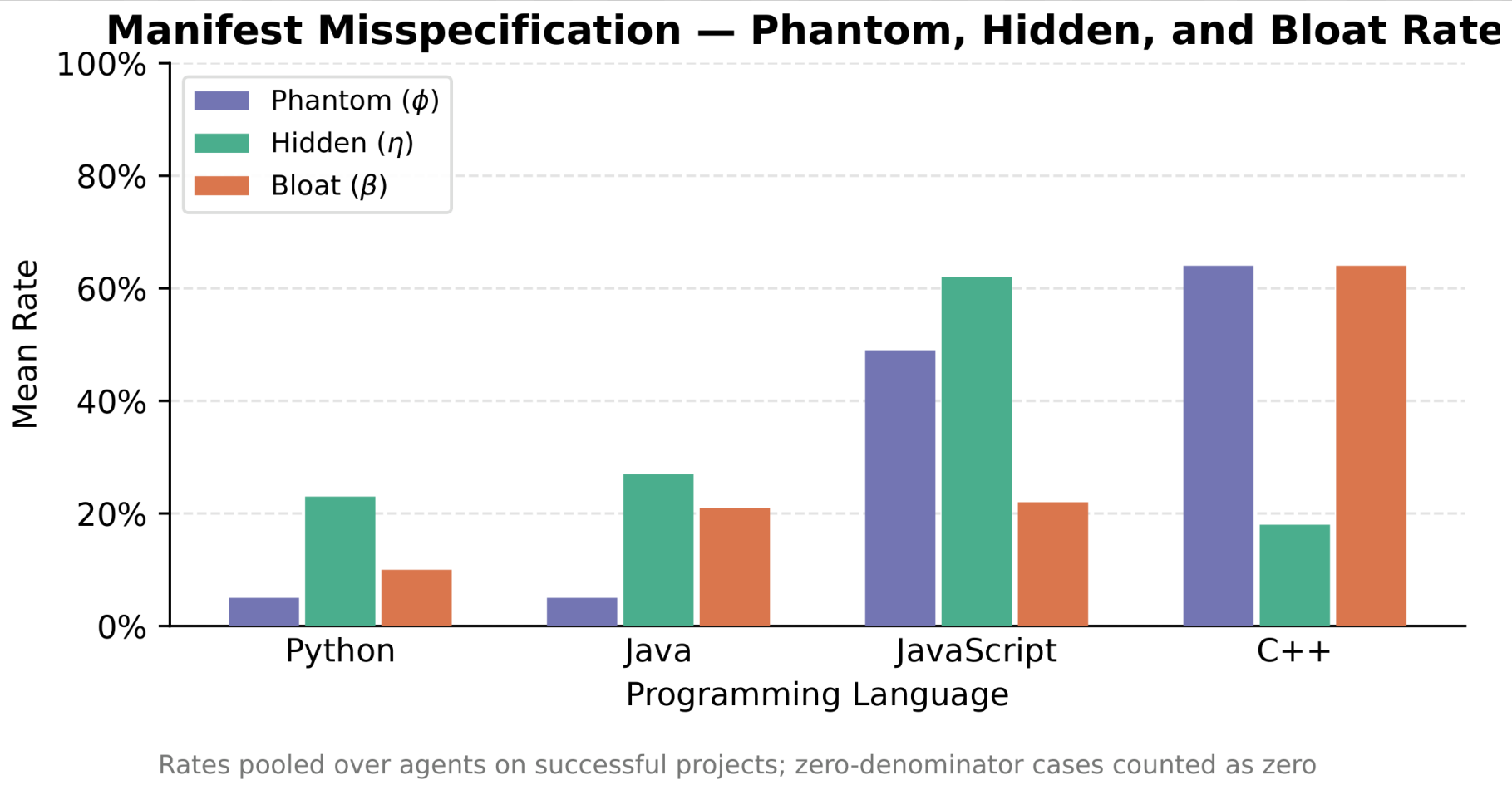
Generation quality, repair capability, and specification quality are evaluated separately.

First-attempt success vs. success after repair



After repair, Claude reaches 100% in all four languages; the lowest final rate is 94%.

Phantom, hidden, and installed-but-unused dependencies



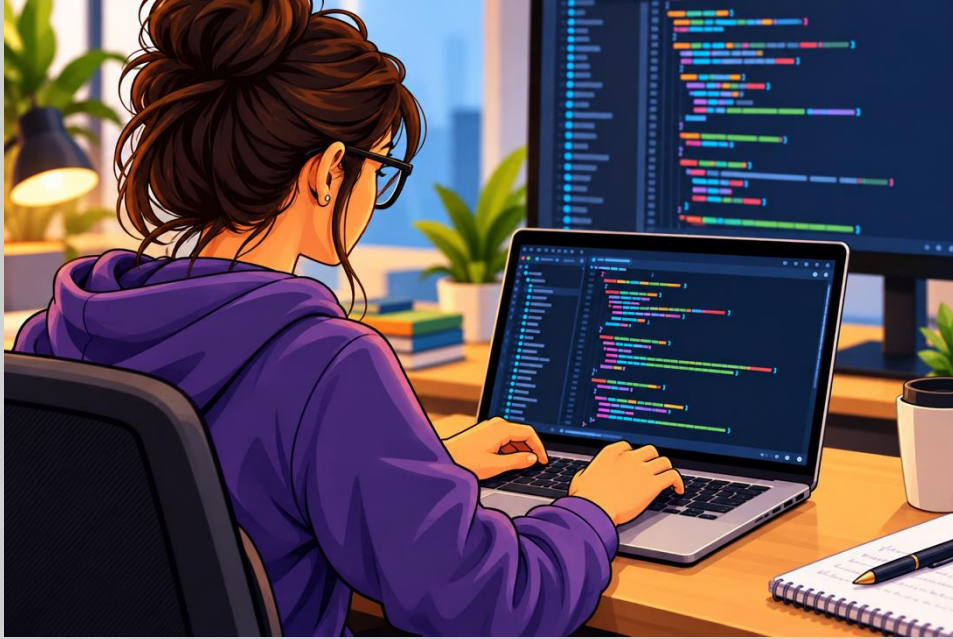
JavaScript: about 49% phantom, 62% hidden, and 22% bloat.

If an AI coding agent tells you what its software needs, how accurate is that specification?

Not reliably—the declared environment often diverges substantially from the environment actually used.

Vangala, B.P, Malik, T., (2026). “Code That Works, Environments That Don’t: Measuring Environment Reproducibility in AI-Generated Software”
Forthcoming in AAAI Magazine.

Let's take AI out of the picture.







Sometimes but not reliably

Let's take AI out of the picture.

The specification gap predates AI

D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden Technical Debt in Machine Learning Systems,” *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, pp. 2503–2511, 2015.

2016-2026

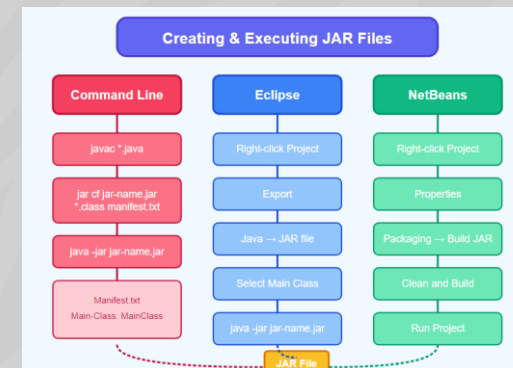
What is the environment of a computation?

```
requirements.txt
You, 17 minutes ago | author (You)
1 keras==2.9.0rc0
2 #numpy==1.22.3
3 Pillow==9.3.1
4 scikit-image==0.19.2
5 streamlit==1.7.0
6 face_detection==0.2.2
7 opencv-python-headless==4.5.5.64
8 streamlit-drawable-canvas==0.8.0
9 click==8.0.4
10 tensorflow-cpu
11 #tensorflow==2.9.0
12 click==8.0.4
13 torch==1.7
14 numpy<1.21 # numba requires numpy<1.21,>=1.17
15 #opencv-python
16 torchvision
17 scipy
18 tqdm
19 #matplotlib==3.3.4
20 facefix==0.2.0
21 lmdb
22 pyyaml
23 tb-nightly
24 yapf
```

```
102:42:001 Cloned repository...
102:42:001 Pulling code changes from Github...
102:42:071 Processing dependencies...

-- pip

Collecting keras==2.9.0rc0
  Downloading keras-2.9.0rc0-py3-none-any.whl (11.6 MB)
    Collecting numpy==1.22.3
    Downloading numpy-1.22.3-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (16.8 MB)
    Downloading Pillow==9.3.1
    Downloading Pillow-9.3.1-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (4.3 MB)
    Downloading scikit-image==0.19.2
    Downloading scikit-image-0.19.2-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (14.8 MB)
    Downloading streamlit==1.7.0
    Downloading streamlit-1.7.0-py3-none-any.whl (19.9 MB)
    Downloading face_detection==0.2.2
    Downloading face_detection-0.2.2.tar.gz (20 KB)
    Preparing metadata (setup.py): started
    Preparing metadata (setup.py): finished with status 'done'
    Collecting opencv-python-headless==4.5.5.64
    Downloading opencv-python-headless-4.5.5.64-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (47.8 MB)
    Downloading tensorflow-cpu==2.9.0
    Downloading tensorflow-cpu-2.9.0-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (207.4 MB)
    Downloading click==8.0.4
    Downloading click-8.0.4-py3-none-any.whl (97 KB)
    Downloading torch==1.7
    Downloading torch-1.7.0-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (41.6 MB)
    Downloading torchvision==0.11.0
    Downloading torchvision-0.11.0-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (16.9 MB)
    Downloading scipy==1.4.1
    Downloading scipy-1.4.1-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (41.6 MB)
    Downloading tqdm==4.64.1
    Downloading tqdm-4.64.1-py3-none-any.whl (78 KB)
    Downloading pyyaml==6.0
    Downloading pyyaml-6.0-cp38-cp38-manylinux_2_17_x86_64_muslmanylinux2014_x86_64.whl (559 KB)
    Downloading tb-nightly==2.10.0a2022.5.4
    Downloading tb-nightly-2022.5.4-py3-none-any.whl (135 KB)
    Downloading yapf==0.32.0
    Downloading yapf-0.32.0-py3-none-any.whl (220 KB)
```



Environment is declared software

But declarations are difficult to obtain

Q. Pham, T. Malik, I. Foster, R. Di Lauro and R. Montella. SOLE: Linking Research Papers with Science Objects. In International Provenance and Annotation Workshop (IPAW), 2012

Q. Pham, T. Malik and I. Foster. Using Provenance for Repeatability. In USENIX 5th Workshop on Theory and Practice of Provenance, (TaPP), 2013.



<https://bit.ly/4jwnU0/sciunit>

Sciunit: From Declarative Recipes to Application Virtualization

Core Idea: Create a package *automatically* with code, data and environment

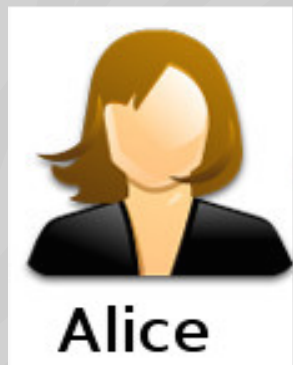
Mechanism: Passive system-call tracing (ptrace) to capture the exact closure of libraries, configs, and binaries into an immutable object.

Q. Pham, T. Malik, B. Glavic and I. Foster. Light-weight Database Virtualization. In IEEE 31st International Conference on Data Engineering (ICDE), 1179-1190, 2015.

D.H. Ton That, G. Fils, Z. Yuan and T. Malik. Sciunits: Reusable Research Objects. In IEEE 13th International Conference on e-Science (eScience), 374-383, 2017.

A. Youngdahl, D.H. Ton That and T. Malik. Scilnc: A Container Runtime for Incremental Recomputation. In IEEE 15th International Conference on eScience (eScience), 291-300, 2019.

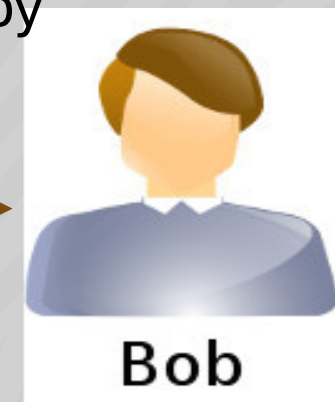
J. Chuah, M. Deeds, T. Malik, Y. Choi and J. Goodall. Documenting Computing Environments for Reproducible Experiments. In Parallel Computing: Technology Trends (PARCO), 756-765, 2020.



sciunit exec myexperiment.py



sciunit repeat myexperiment.py



No need for requirements.txt, documentation, or any install instructions



<https://bit.ly/4jwnU0/sciunit>

Sciunit: From Declarative Recipes to Application Virtualization

Core Idea: Create a package *automatically* with code, data and environment

Mechanism: Passive system-call tracing (ptrace) to capture the exact closure of libraries, configs, and binaries into an immutable object.

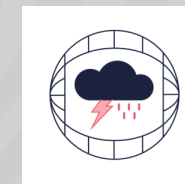
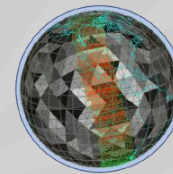
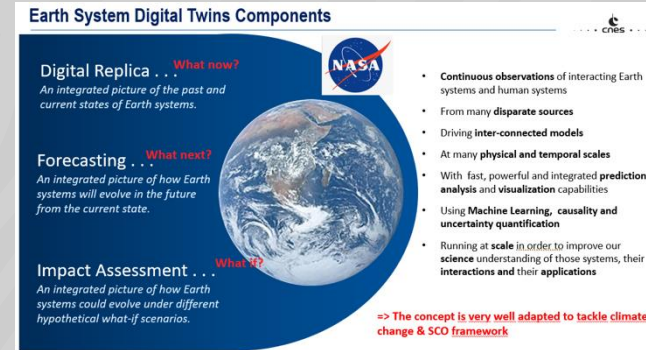
Q. Pham, T. Malik, B. Glavic and I. Foster. Light-weight Database Virtualization. In IEEE 31st International Conference on Data Engineering (ICDE), 1179-1190, 2015.

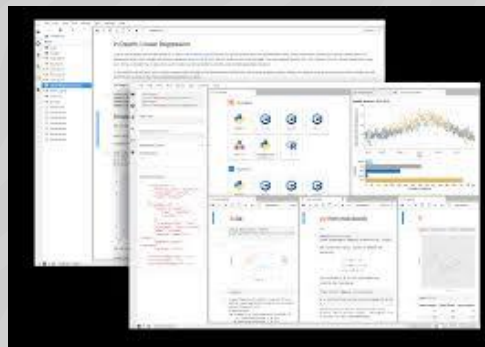
D.H. Ton That, G. Fils, Z. Yuan and T. Malik. Sciunits: Reusable Research Objects. In IEEE 13th International Conference on e-Science (eScience), 374-383, 2017.

A. Youngdahl, D.H. Ton That and T. Malik. SciInc: A Container Runtime for Incremental Recomputation. In IEEE 15th International Conference on eScience (eScience), 291-300, 2019.

J. Chuah, M. Deeds, T. Malik, Y. Choi and J. Goodall. Documenting Computing Environments for Reproducible Experiments. In Parallel Computing: Technology Trends (PARCO), 756-765, 2020.

Sciunit Adoption





Environment has memory

Execution state ensures reproducibility

N. Manne, S. Satpati, T. Malik, A. Bagchi, A. Gehani and A. Chaudhary. CHEX: Multiversion Replay with Ordered Checkpoints. In Proceedings of the Very Large Databases (PVLDB), 15(6): 1297-1310, 2022. doi: 10.14778/3514061.3514075 2022



<https://bit.ly/4jwnU0/FlinC>

FlinC: Virtualizing the Living Kernel of Interactive Notebooks

Core Idea: Capturing state when execution is non-linear and exploratory.

Mechanism: Notebook audit and repeat kernels with checkpoints and passive system-call tracing (ptrace) to capture the exact closure of libraries, configs, and binaries into an immutable object.

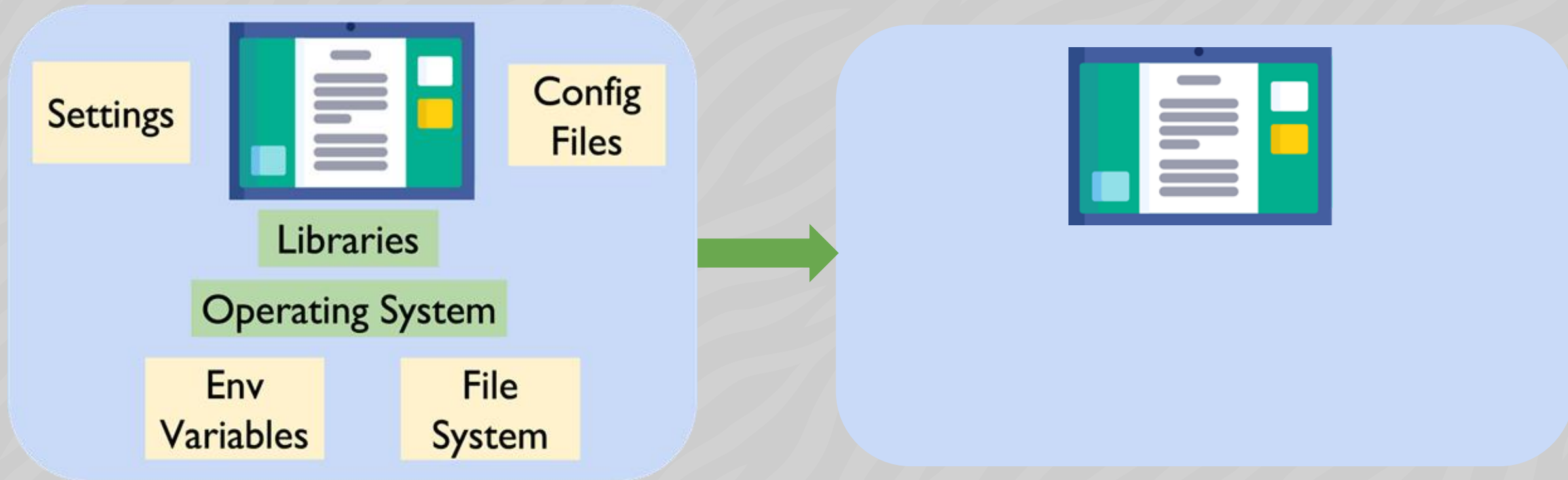
R. Ahmad, N. Manne and T. Malik. Reproducible Notebook Containers using Application Virtualization. In IEEE eScience, 1-10, 2022. doi: 10.1109/eScience55777.2022.00015 [Best Student Paper]

T. Azaz, R. Ahmad, Md S. Islam, D. Thain, and T. Malik. Efficiently Reproducing Distributed Workflows in Notebook-based Systems. In the 26th IEEE International Symposium on Cluster, Cloud, and Internet Computing (CCGrid), 2026

Notebook Environment is Not Shared

Host Machine

Target Machine

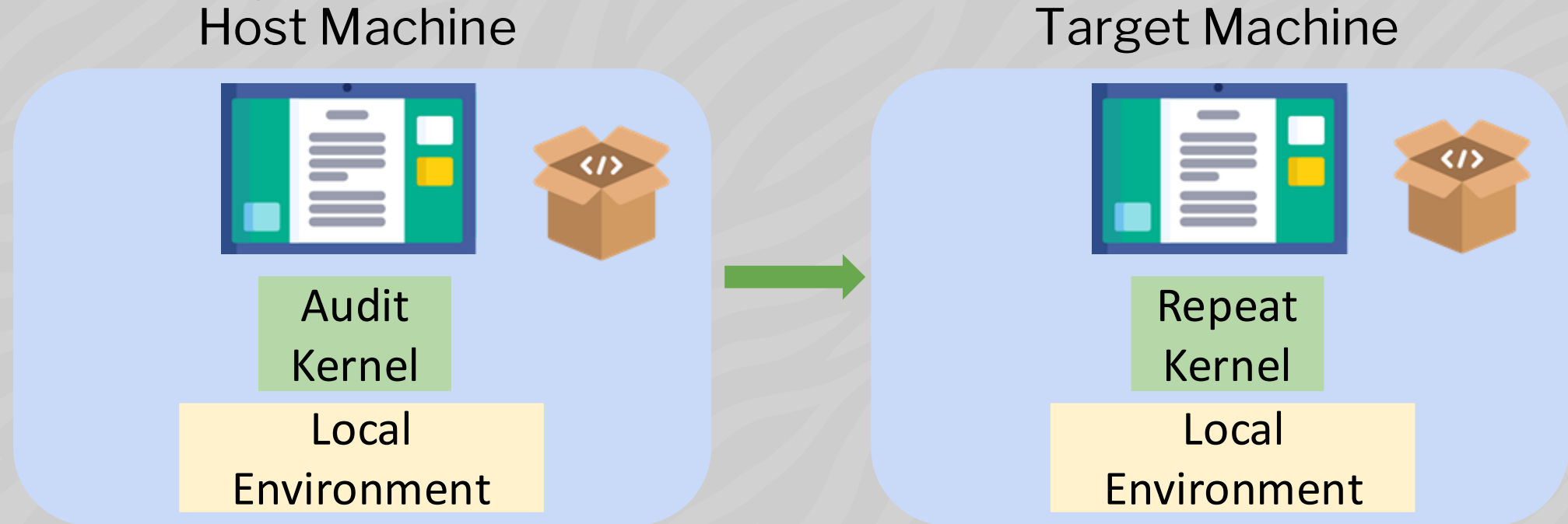


Extend AV to Notebook Client-Server Architecture

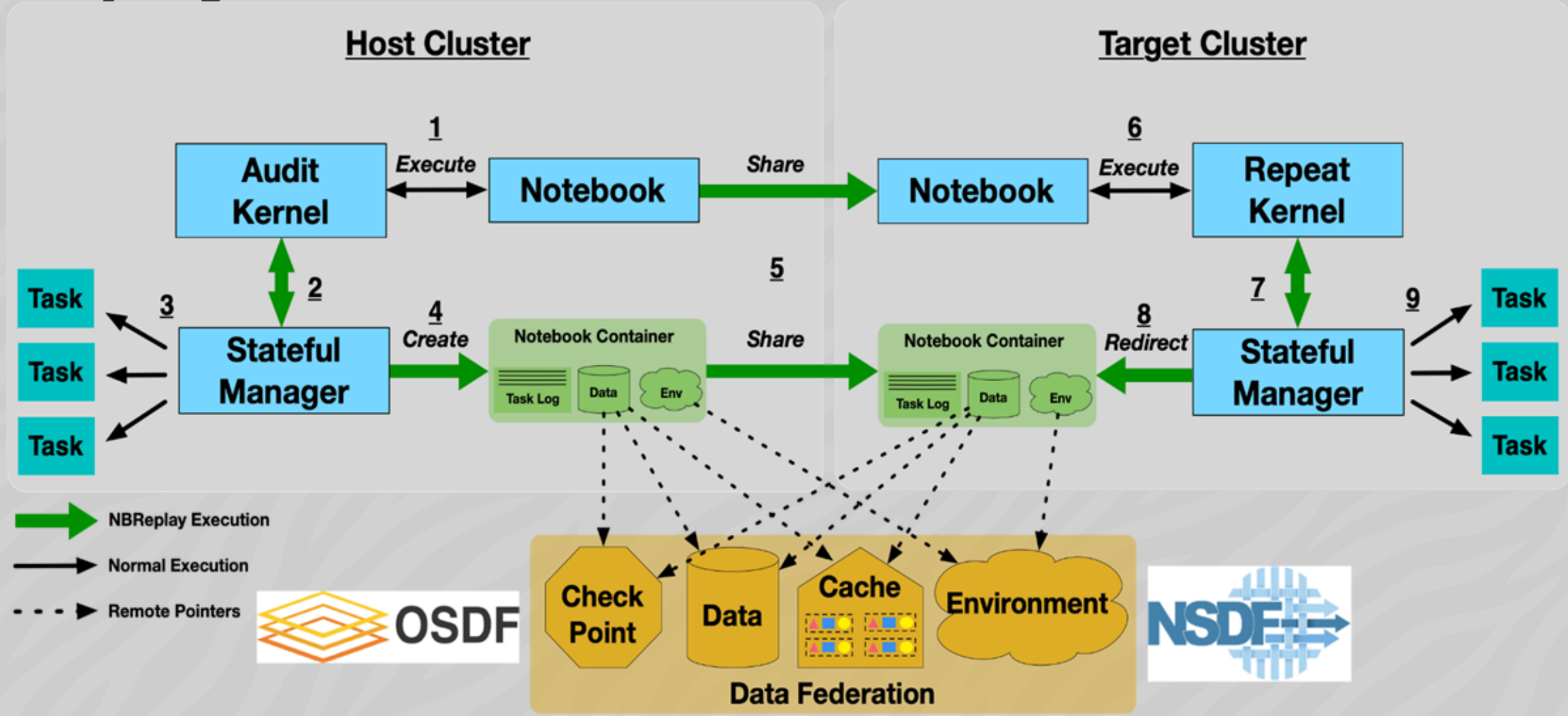
Creates two additional kernels:

Audit kernel: executes and audits notebook code

Repeat kernel: repeats the containerized code



NBReplay Architecture and Workflow



Environment includes lots of data

Data is often read-only and large

C. Niddodi, A. Gehani, T. Malik, S. Mohan, and M. Rilee, IOSPreD: I/O Specialized Packaging of Reduced Datasets and Data-Intensive Applications for Efficient Reproducibility, In IEEE Access, Vol. 11, 2023. doi: 10.1109/MCSE.2021.3096678.



<https://bit.ly/4jwnU0/FIinc>

Kondo: Provenance-Driven Data Debloating

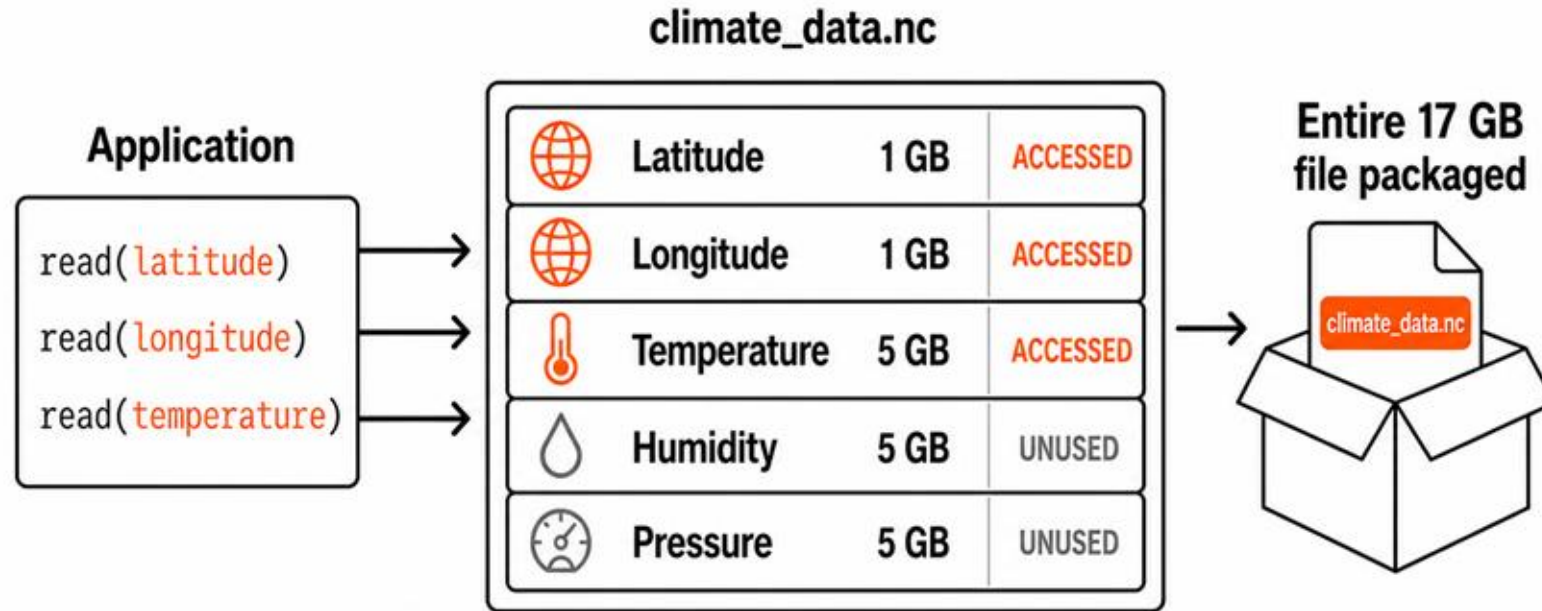
Core Idea: Capturing full context without terabytes of redundant intermediate files.

Mechanism: asks what data *actually* contributes to the computation and uses data provenance to retain what is necessary for replay and verification.

1. R. Atiq, A. Gehani, T. Malik, F. Zaffar. Semantically-Aligned Data Carving for Robust Reproducibility. In 10th ACM Practice and Experience in Advanced Research Computing (PEARC), 2026.
2. R. Atiq, A. Gehani, T. Malik, F. Zaffar. SCALPEL: Structured Content Access Logging and Pruning for Efficient Layouts. In IEEE International Conference on eScience (eScience), 2025.
3. R. Tikmany, A. Modi, R. Atiq, M. Reyad, A. Gehani, T. Malik. Efficiently Reducing Storage Foot-print in Reproducible Containers via I/O Specialization. In IEEE Cluster, Cloud and Internet Computing (CCGrid), 2024.
4. A. Modi, R. Tikmany, T. Malik, R. Komondoor, A. Gehani and D. D'souza. Kondo: Efficient Provenance-driven Data Debloating. In IEEE International Conference of Data Engineering (ICDE), 2024.

Workflows are Data-intensive

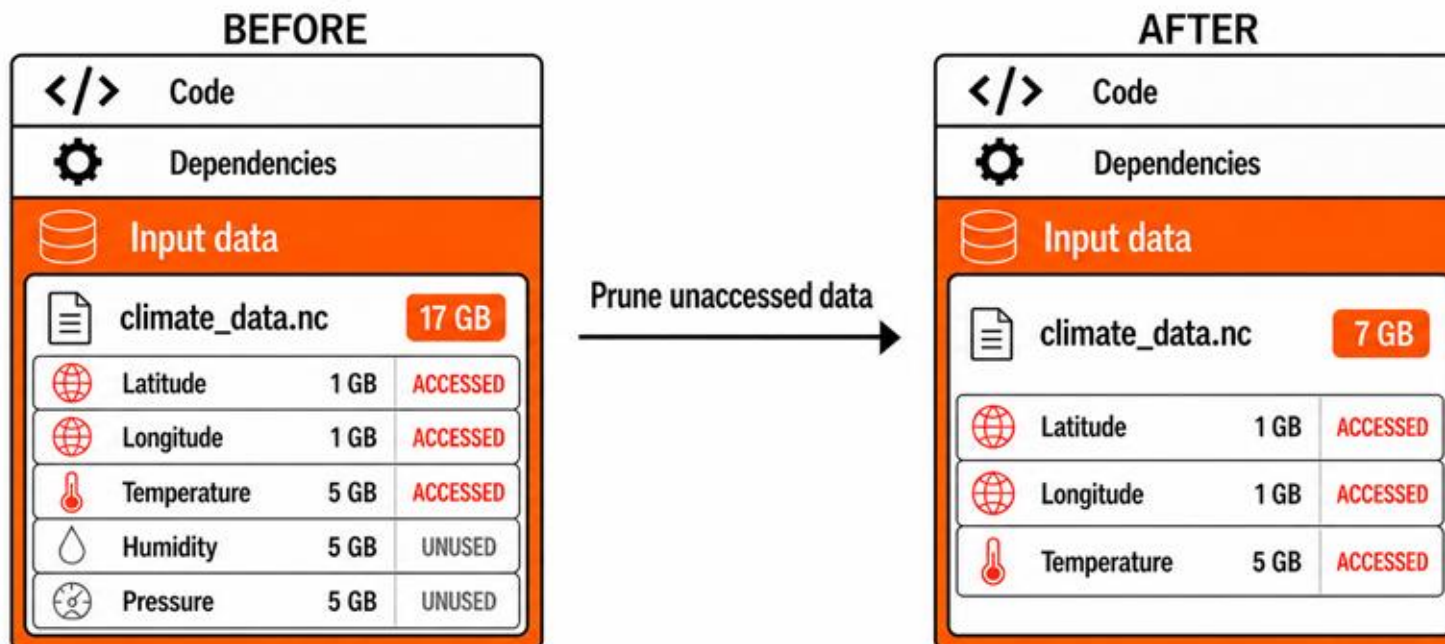
Inside climate_data.nc, the workflow reads 7 GB of 17 GB



Access = 17 GB but used = 7GB

38

Data carving reduces climate_data.nc from 17 GB to 7 GB



Same workflow. Less data to store and share.

Data Reduction Results on Earth Science Workflows

	Name	Original Size	Carved Size	Percentage Reduction
POMD-PF	POMD-PF	862 MB	475 MB	45%
	Feature Database	18 GB	4.4 GB	76%
	Kriging OMI Level 2	579 MB	34 MB	94%
	Kriging VIIRS	75 GB	10 GB	87%
EcoPro	Tree Mortality	10.3 GB	6.7 GB	32%
	Kelp Forest Projection	7.8 GB	4.3 GB	45%



Table 2: Size reduction achieved by the carving tool across a range of real-world scientific applications.

	Name	Original	Execution Mode	Re-execution Mode
POMD-PF	POMD-PF	64 s	64 s	61 s
	Feature Database	2731 min	3017 min	2856 min
	Kriging OMI Level 2	5 s	212 s	8 s
	Kriging VIIRS	2710 min	2781 min	2633 min
EcoPro	Tree Mortality	449 s	506 s	491 s
	Kelp Forest Projection	437 min	460 min	406 min



Environment is not restricted to one machine

It is spread across several heterogeneous, distributed nodes



Fidelis: Auditing Distributed Workflows without Cluster Overhead

Core Idea: Whole-environment capture across thousands of distributed workers without decreasing cluster throughput

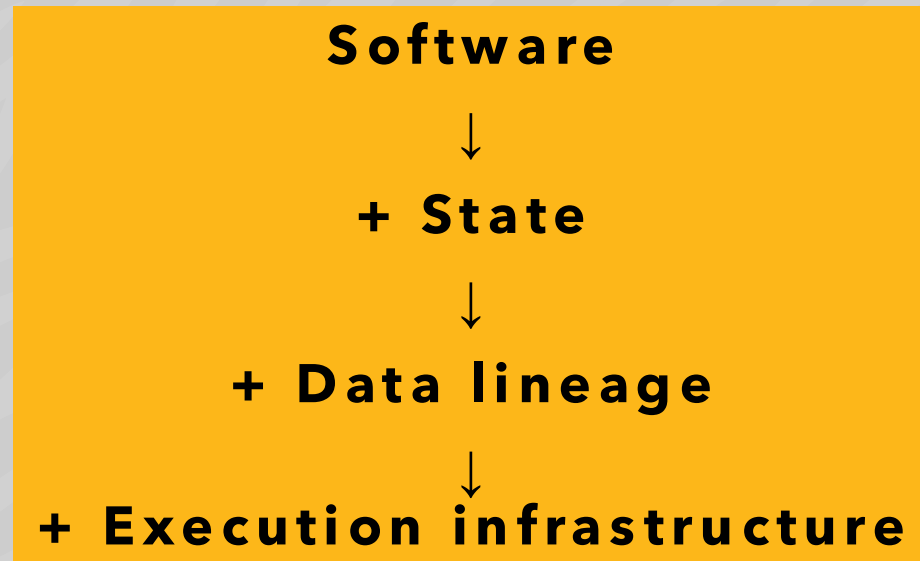
Mechanism: Context similarity exploiting the fact that distributed tasks share structural commonality. Uses execution **context reuse & minimized context-switching** providing up to 93% reduction in auditing overhead and reduction in replay time across heterogeneous clusters.

R. Ahmad, T. Azaz, D. Thain, T. Malik, Fidelis: Exploiting Task Similarity for Low-overhead Distributed Workflow Audit and Replay, Forthcoming in Middleware. 2026

What exactly is in the environment?



What exactly is the environment?

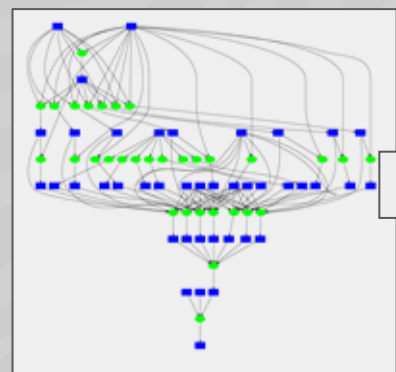


A Backpack¹

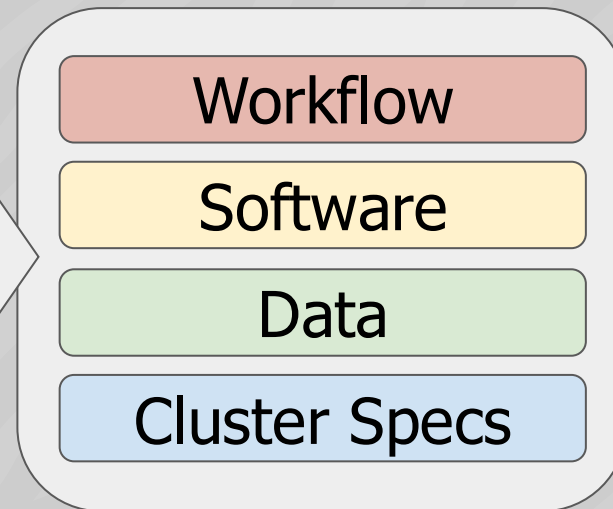


Term credit: *Douglas Thain, University of Notre Dame*

A Backpack¹



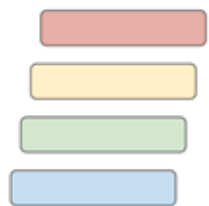
Workflow



Backpack

A **backpack** is structured specifications of all its dependencies — data, software, and computational resources—needed for seamless execution across diverse, heterogeneous environments.

Term credit: *Douglas Thain, University of Notre Dame*



floability



This work was supported in part by NSF grant
2411436 CSSI Frameworks: From
Notebook to Workflow and Back Again

floability.github.io

```
conda install -c conda-forge floability
```



UNIVERSITY OF
NOTRE DAME



Mizzou
University of Missouri



ILLINOIS



DEPAUL UNIVERSITY

Example: DV5 CMS Analysis Application

<http://dx.doi.org/10.1109/SC41406.2024.00068>



Kevin Lannon Connor Moore

```
In [3]: def analysis(events):
File display
warnings.filterwarnings("ignore", module="coffea.*")

dataset = events.metadata["dataset"]
events["PFCands", "pt"] = events.PFCands.pt
events["PFCands", "pt"] = events.PFCands.pt

cut_to_fix_softdrop = ak.num(events.FatJet.constituents.pf, axis=2)
events = events[ak.all(cut_to_fix_softdrop, axis=1)]

trigger = ak.zeros_like(ak.firsts(events.FatJet.pt), dtype="bool")
for t in triggers["2017"]]:
    if t in events.HLT.fields:
        trigger = trigger | events.HLT[t]
    trigger = ak.fill_none(trigger, False)
events["FatJet", "num_fatjets"] = ak.num(events.FatJet)

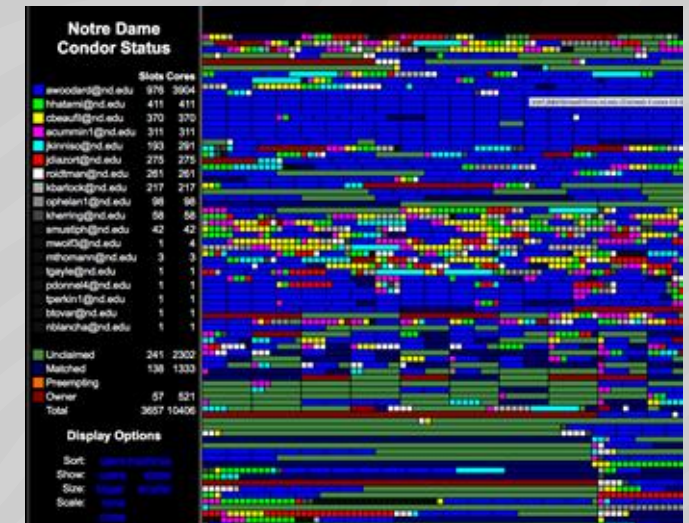
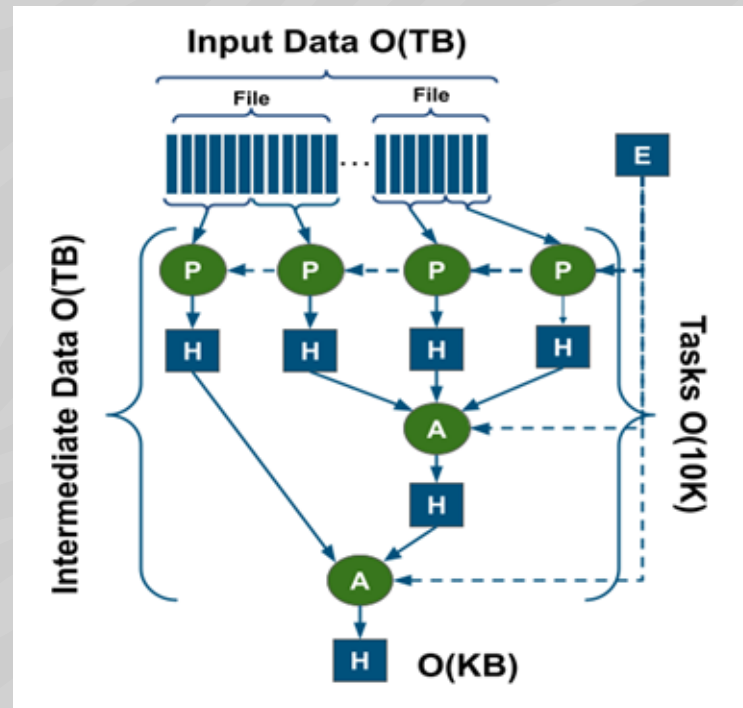
goodmuon = (
    (events.Muon.pt > 10)
    & (abs(events.Muon.eta) < 2.4)
    & (events.Muon.pfRelIso04_all < 0.25)
    & events.Muon.looseId
)

nmuons = ak.sum(goodmuon, axis=1)
```

```
In [5]: m = DaskVine(
[9123, 9128],
name=f"{os.environ.get('VINE_MANAGER_NAME')}",
)
```

```
In [6]: computed = dask.compute(
tasks,
scheduler=m,
progress_disable=True,
resources={"cores": File display,
resources_mode=None,
})

print("Compute done.")
```



Consumes 1.5TB Data
Submits 17K Tasks
Uses 2400 cores, 200 nodes

Example: Aging Dams Simulation Workflow

<https://github.com/floability/floability-cli/tree/distributed-iguide-gis-aging-dams/example/iguide-gis-aging-dams>

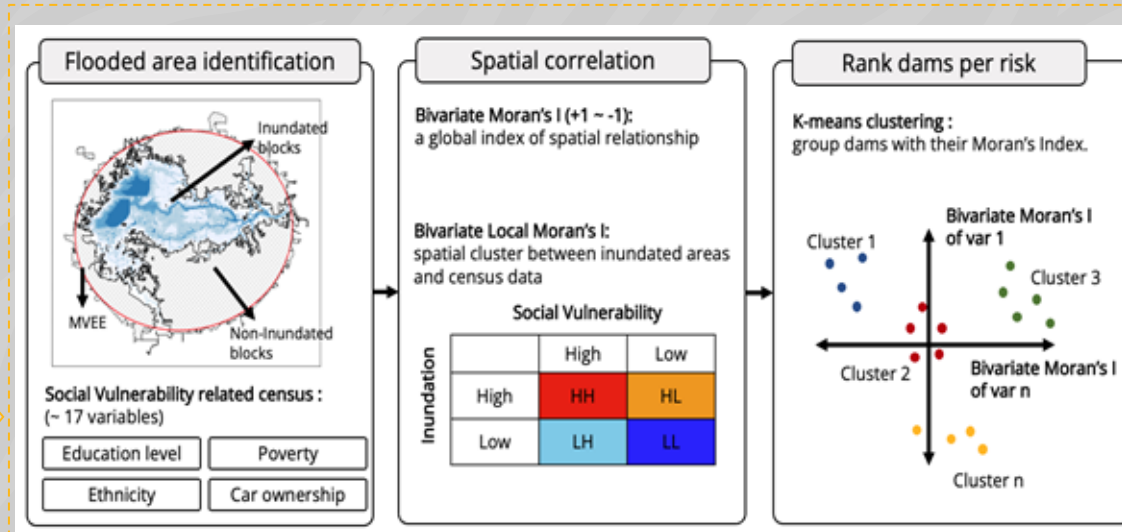


**Furquan
Baig**

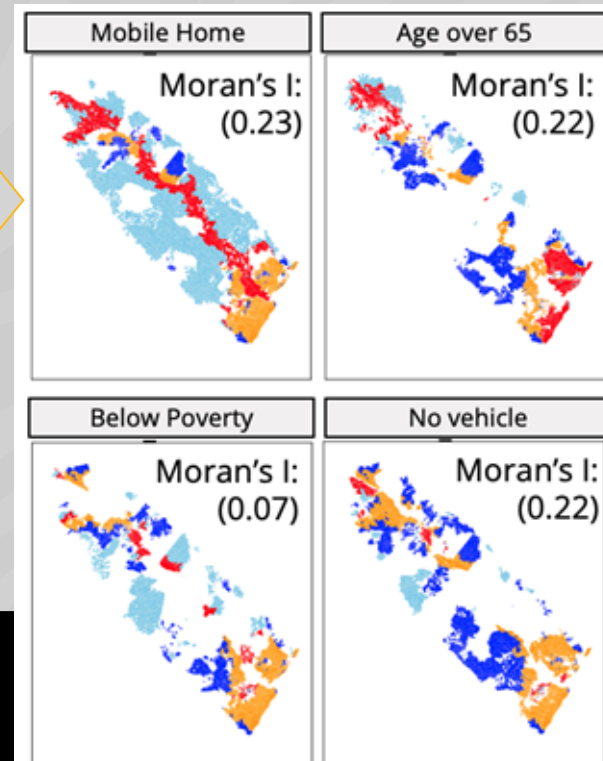
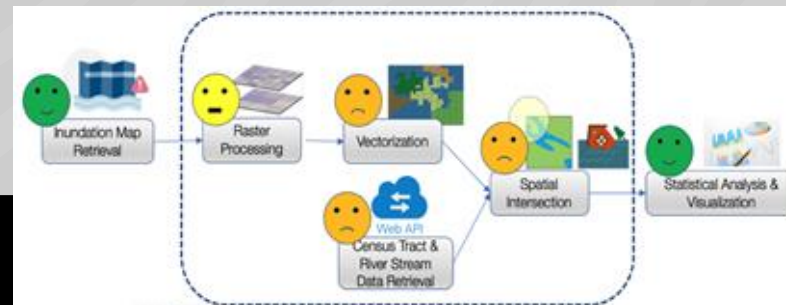


**Shaowen
Wang**

Analyze
Downstream
Impacts of
Dam Failures
to Critical
Infrastructure,
Vulnerable
Populations &
Ecosystems



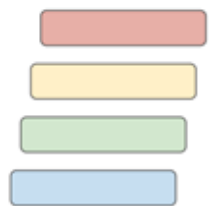
Modularized steps
(Flooded area identification)



Backpack

- workflow
 - ◆ cms-physics-dv5.ipynb
- software
 - ◆ environment.yml
 - ◆ worker-environment.yml
- data
 - ◆ data.yml
 - ◆ samples/qcd/.../nano_mc2017_11.root
 - ◆ samples/diboson/.../nanomc2017_6.root
- compute
 - ◆ compute.yml

<https://github.com/floability-hub>



floability



This work was supported in part by NSF grant
2411436 CSSI Frameworks: From
Notebook to Workflow and Back Again

floability.github.io

```
conda install -c conda-forge floability
```



UNIVERSITY OF
NOTRE DAME



Mizzou
University of Missouri



ILLINOIS



DEPAUL UNIVERSITY

An eScience Vision: as a CI primitive

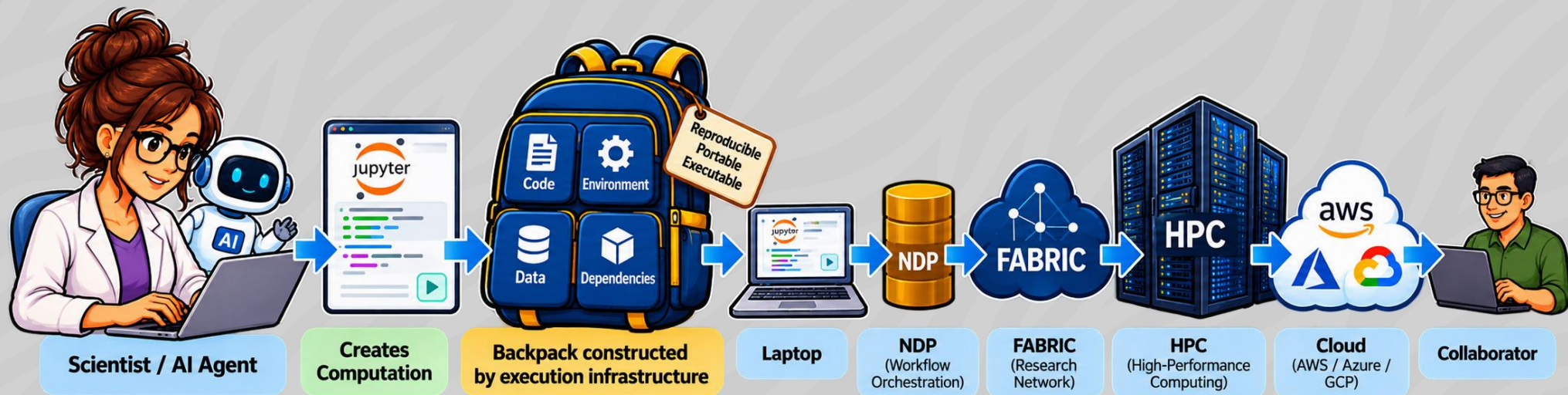
An eScience Vision: as a CI primitive

- Humans were and AI agents are imperfect environment specifiers
 - The difference is only in scale
- Software artifacts can be generated faster than humans can inspect them.
 - Artifacts cannot be responsible about their execution context.
- Instead, the *execution infrastructure* should observe that context and construct the backpack as the computation is created.



Reproducibility Context as a Service

- Environment context becomes a first-class object of cyberinfrastructure, just as data, identity, and computation have become first-class objects.



When human/AI writes the code, who carries the context?

Infrastructure

When AI writes the code, who carries the context?

Infrastructure



Collaborators and Students

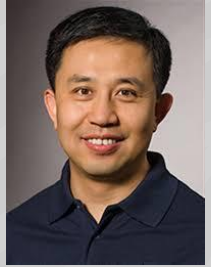
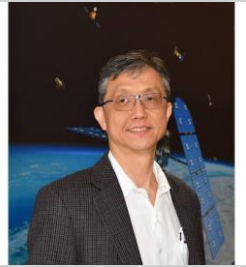
Former Postdocs, Engineers and Students



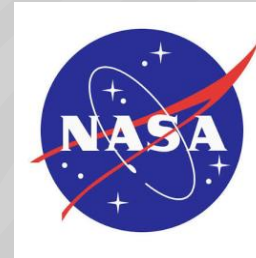
Current Students



Collaborators



- Thanks to our funders





Questions

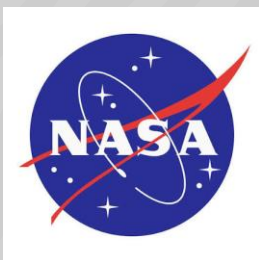
- [Email: tanu@missouri.edu](mailto:tanu@missouri.edu)

- The Radiant Lab

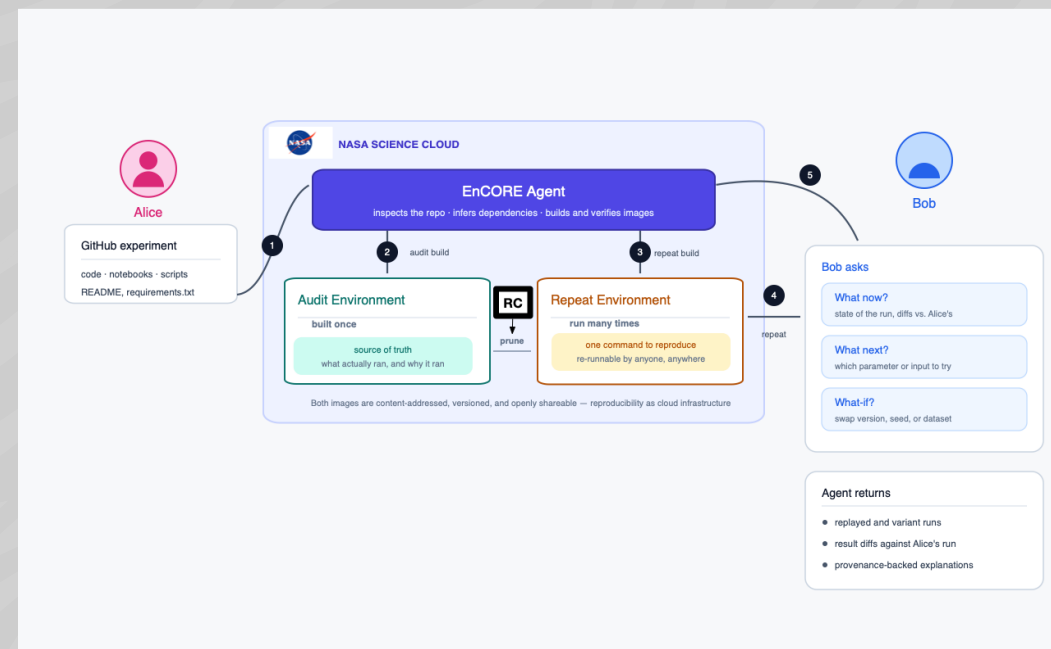
<https://radiant-systems-lab.github.io>

We are looking for post docs and

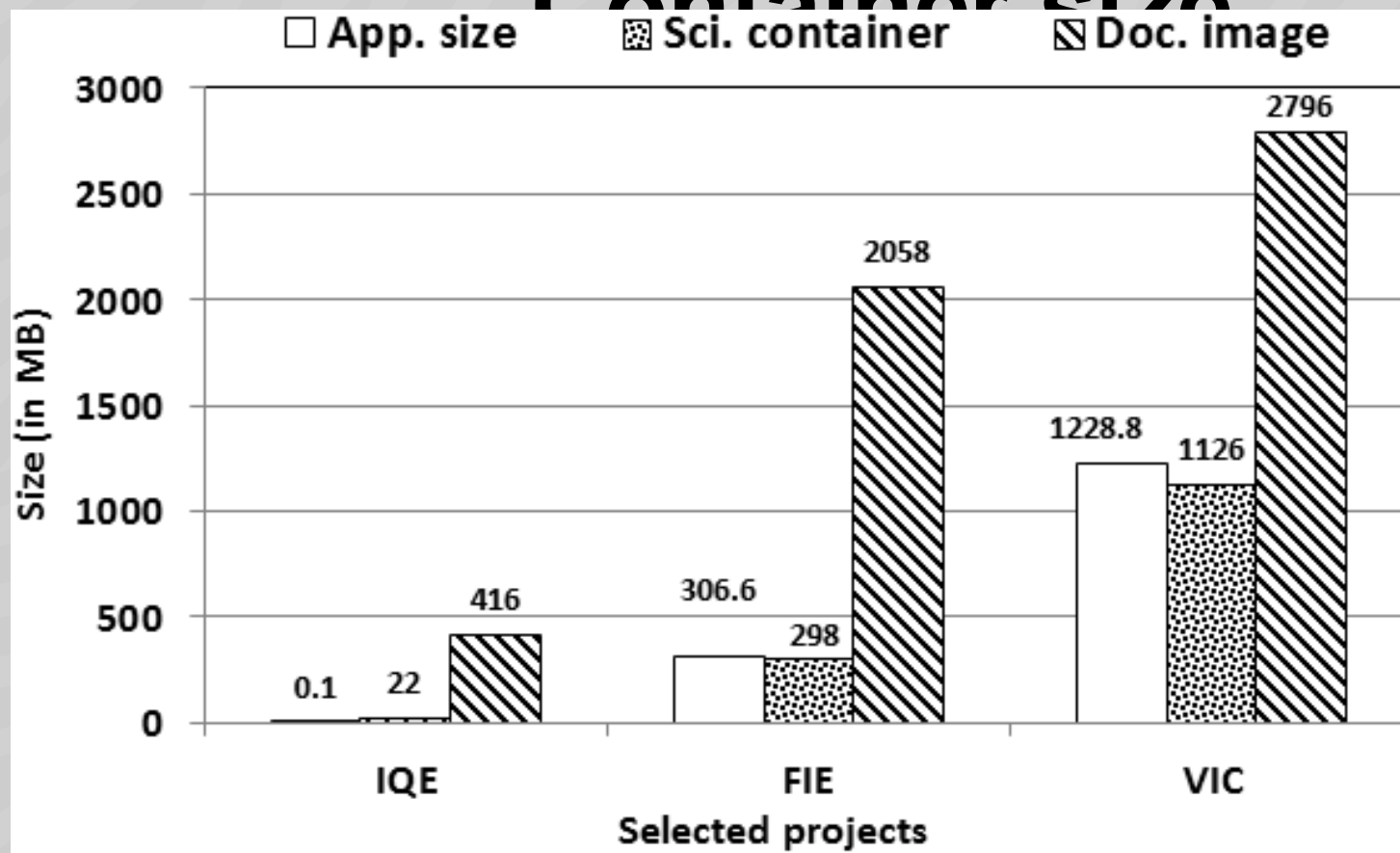
EnCORE – Agent-Powered Reproducible Container Infrastructure for Digital Twins



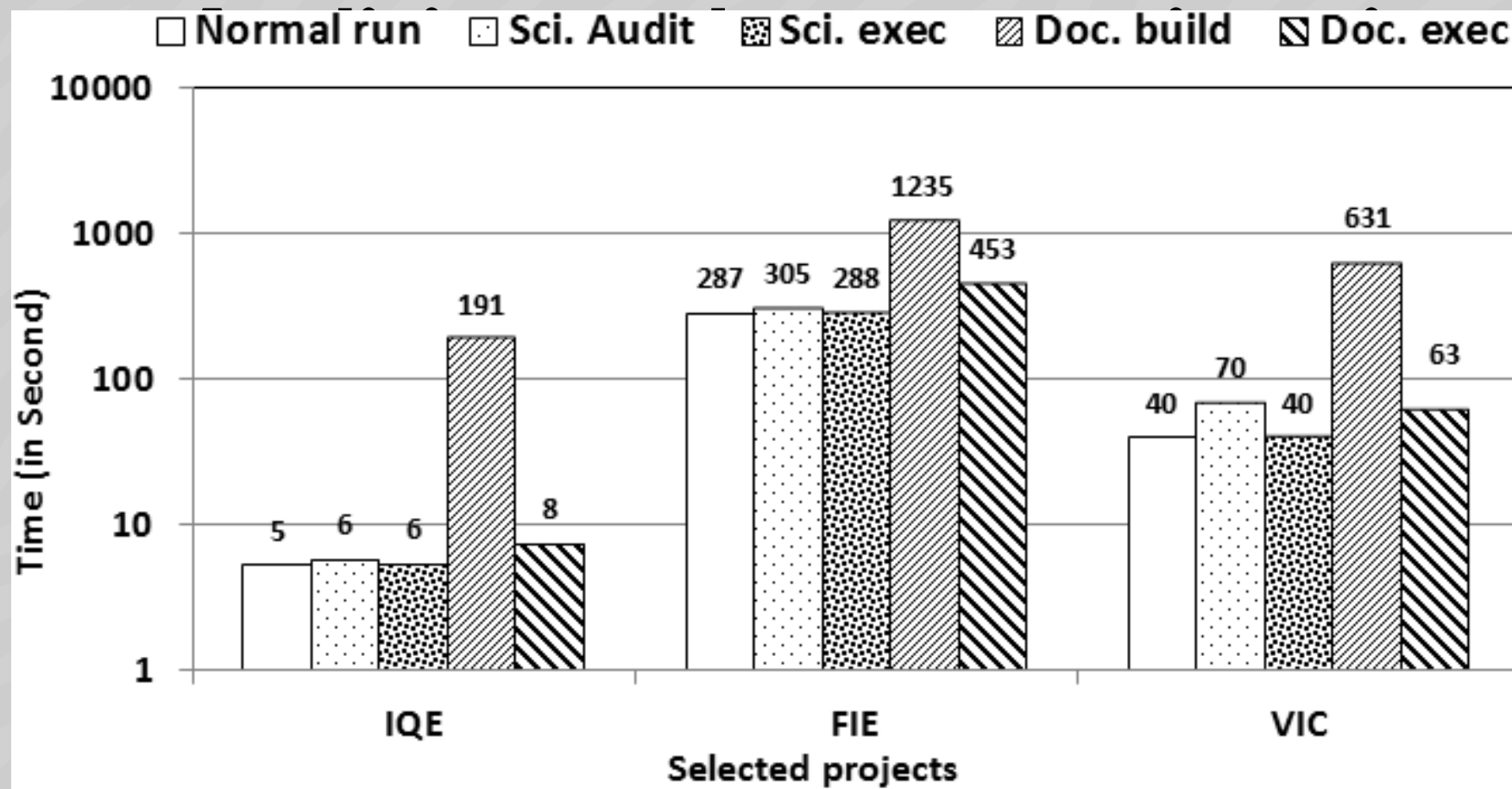
1. Scientist provides a GitHub link
2. EnCORE Agent sets up two environments
3. Audit will create the data-efficient container
4. Repeat will re-execute the container w/o environment setup
5. Users can ask What-If, What Now, What next



Container size



1. Docker containers are 19X, 7X and 2.5X larger than those of Sciunit.
2. Sciunit containers are even slightly smaller than the original application



1. Docker spends 4X and 9X longer time than Sciunit to build FIE and VIC container, 7X and 2.5X larger.

Sample Interaction of Sciunit

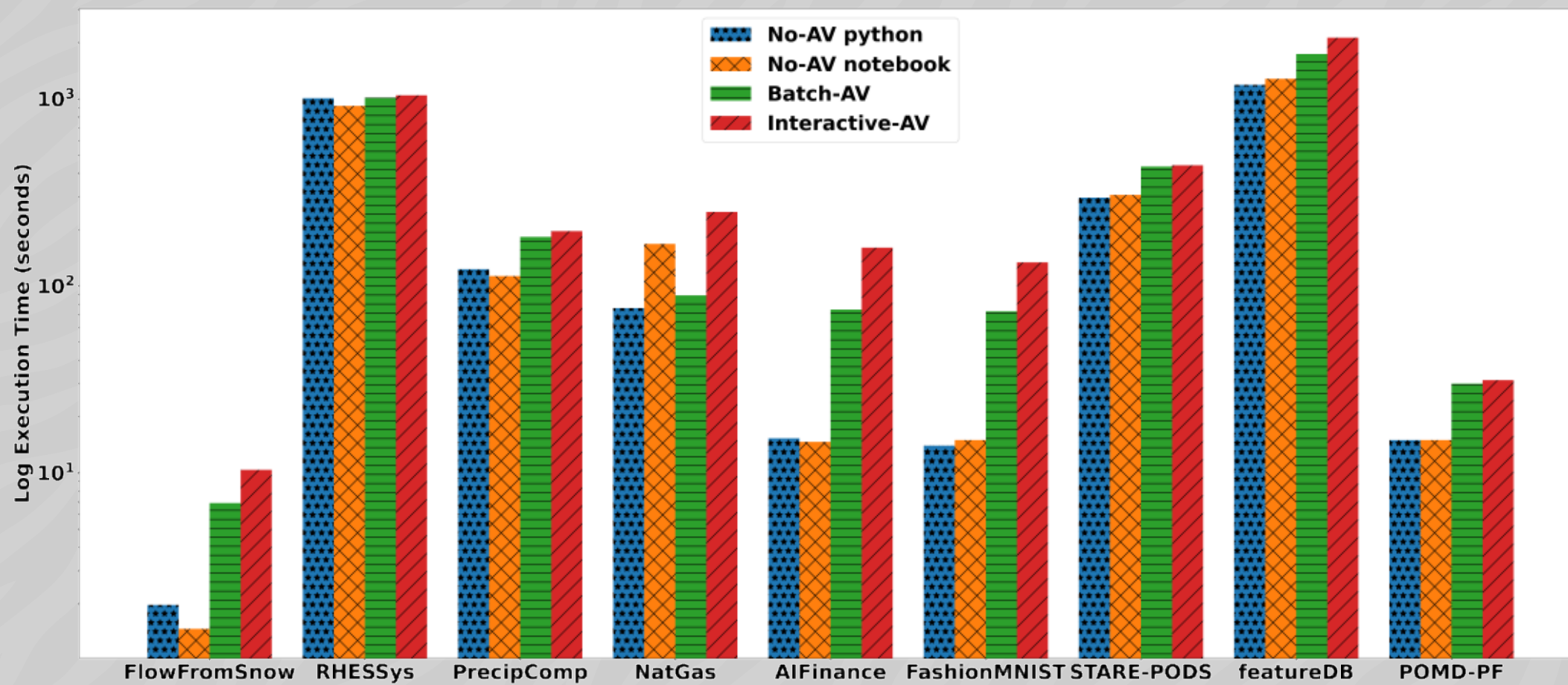
```
1. > sciunit create FIE
2. > sciunit exec ./FIE.sh ./DATA/weather_201710.Rds
    0. Download...
    1. Calculate violation matrix...
    2. Calculate heat map...
    3. Generate model data with ./DATA/weather_201710.Rds...
    4. Apply random forest model...
    5. Evaluation...
3. > sciunit list
    e1 Dec 4 12:44 ./FIE.sh ./DATA/weather_201710.Rds
4. > sciunit show
    id: e1
    sciunit: FIE
    command: ./FIE.sh ./DATA/weather_201710.Rds
    size: 306.6 MB
    started: 2017-12-04 12:44
5. > sciunit push
    ...
    Title for the new article: FIE
    new: 306.6 MB [01:05, 4.72MB/s]
6. > sciunit copy
    mSLLTj#
```

Alice's Computer

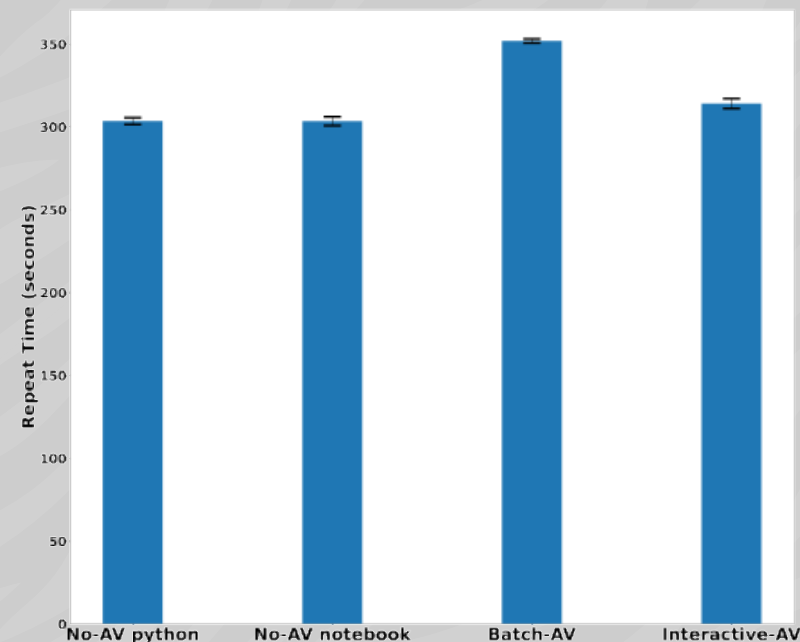
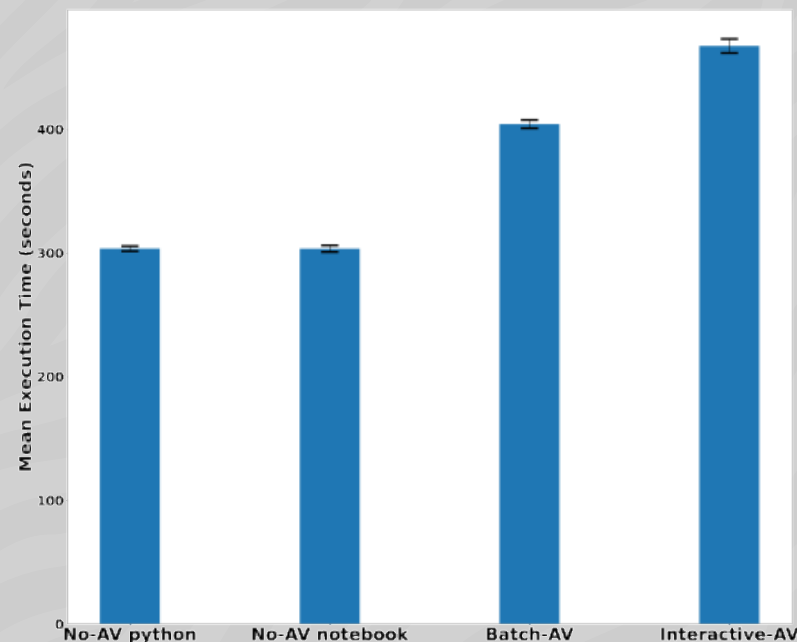
```
1. > sciunit open mSLLTj#
    Opened Sciunit FIE
2. > sciunit list
    e1 Dec 4 12:44 ./FIE.sh ./DATA/weather_201710.Rds
3. > sciunit repeat e1
    ...
    0. Download...
    1. Calculate violation matrix...
    2. Calculate heat map...
    3. Generate model data with ./DATA/weather_201710.Rds
    4. Apply random forest model...
    5. Evaluation...
4. > sciunit given '/tmp/weather_201801.Rds' e1 %
    ...
    1. Generate model data with '/tmp/weather_201801.Rds
    2. Apply random forest model...
    3. Evaluation...
5. > sciunit list
    e1 Dec 4 12:44 ./FIE.sh ./DATA/weather_201710.Rds
    e2 Dec 14 2:44 ./FIE.sh ./tmp/weather_201801.Rds
```

Bob's Computer

Notebook Execution Time

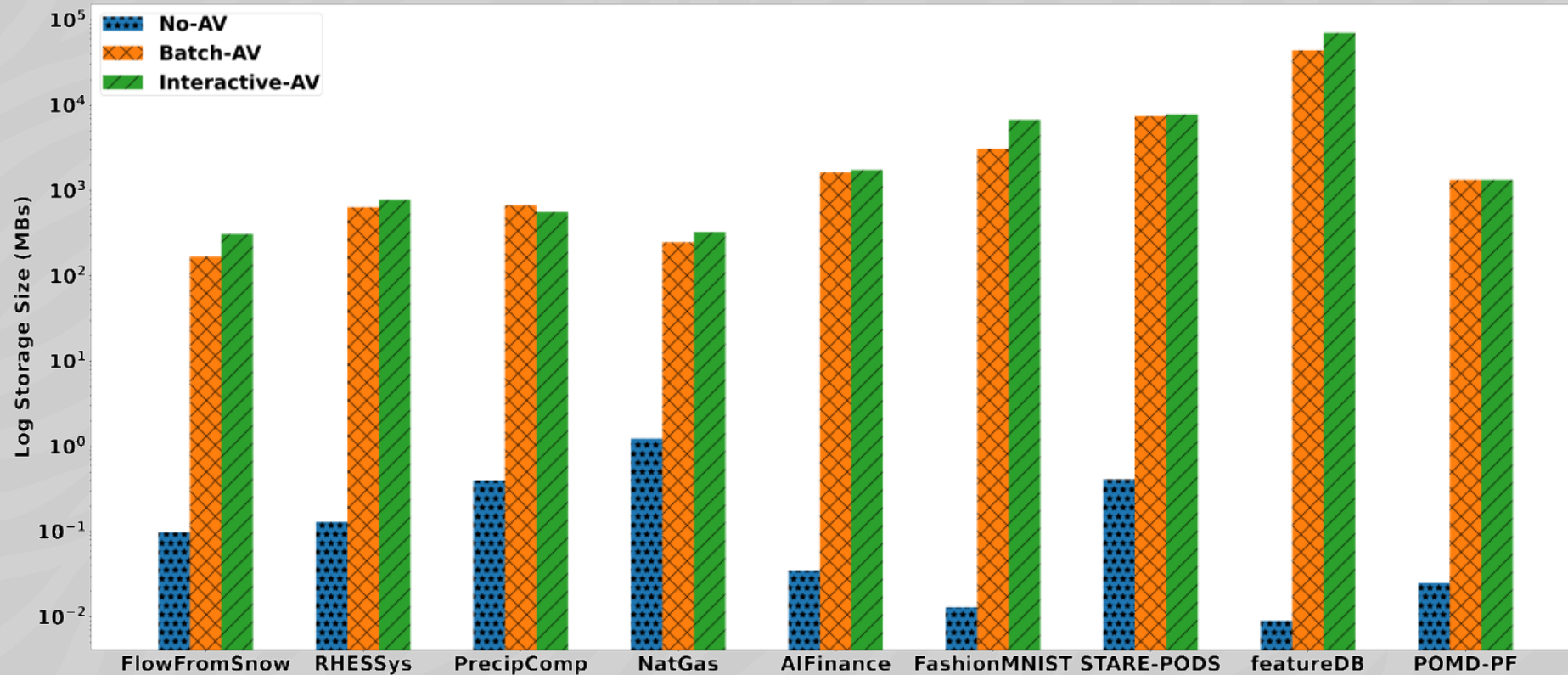


Average Execution Time



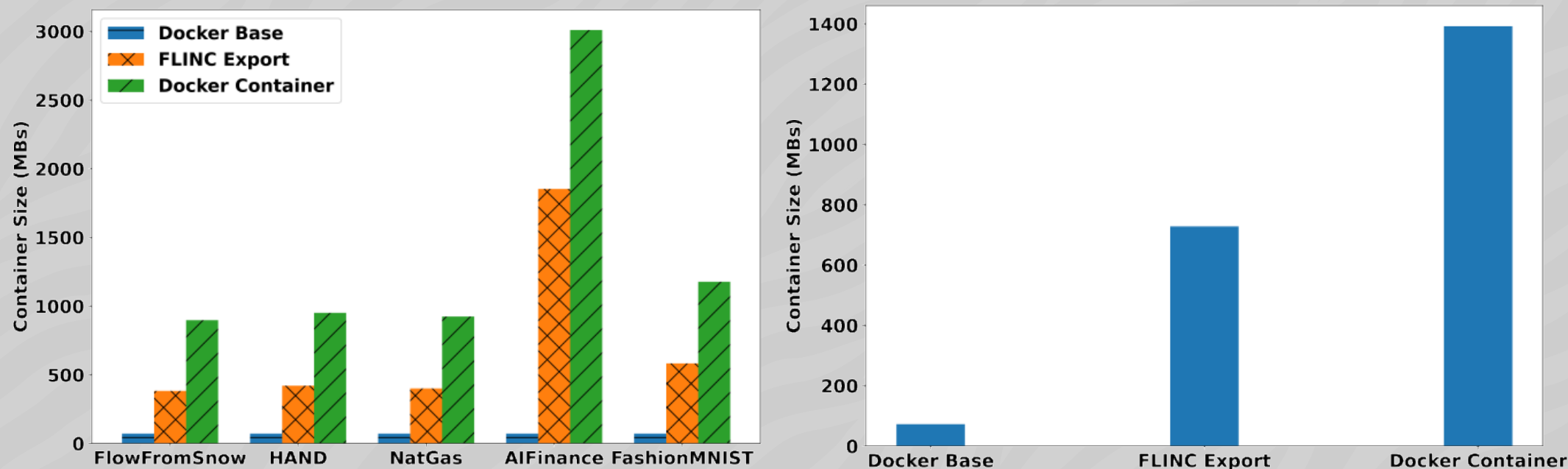
- Audit: About 16% overhead of interactive AV than batch AV.
- Repeat: Interactive-AV method adds an overhead of only around 3.5% compared to the actual notebook execution, but batch-AV is more expensive.
 - Almost 11% less than the time it takes to repeat the same notebook using batch-AV

Storage size of notebook containers



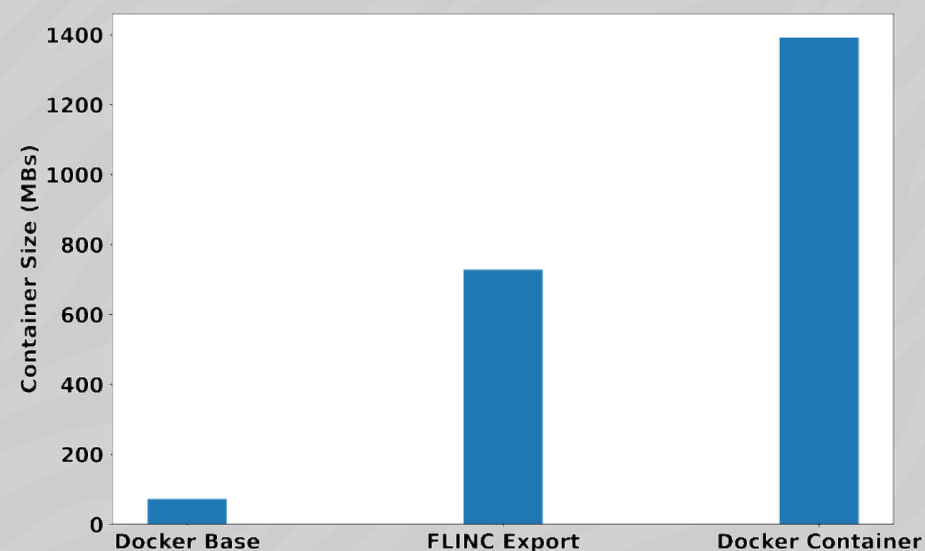
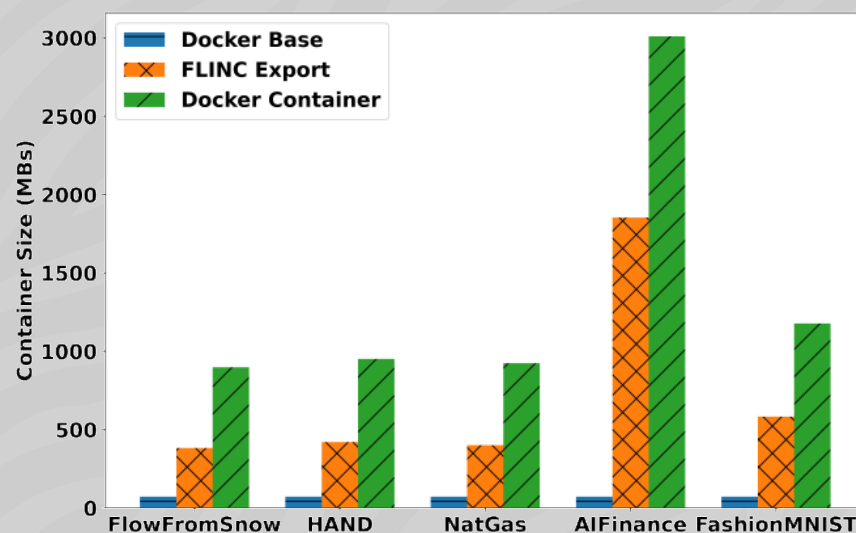
- 34% larger in size as compared to the containers created using Batch-AV.
- Intermediate data cached by notebook execution accounts for the primary increase in size.

Notebook Containers Vs Docker



46% less time than Docker to create a new virtual environment on the target system and install the required dependencies.

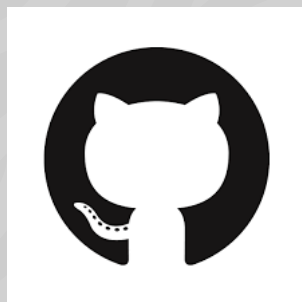
Notebook Containers Vs Docker (2)



- Docker containers are, on average, 91% larger in size for all our notebooks compared to the containerized environment created by FLINC.



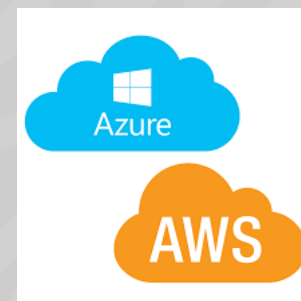
Reproducibility ecosystem



Github



Package managers



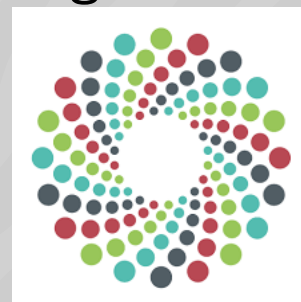
Sharing images via the cloud



Zenodo.org



OpenData.gov



Figshare



Docker.com

An introduction to **Docker** for reproducible research

C Boettiger - ACM SIGOPS Operating Systems Review, 2015 - dl.acm.org



Mizzou®

'26 eScience





Docker: Using containers from build to run



Build

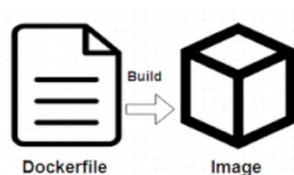


Ship



Run

- Build images that captures applications requirements.
- Manually commit or use a recipe file.
- Push an image to DockerHub, a hosted registry, or a private Docker Registry.
- Share Images
- Use Docker Engine to pull images down and execute a container from the image.



Mizzou®

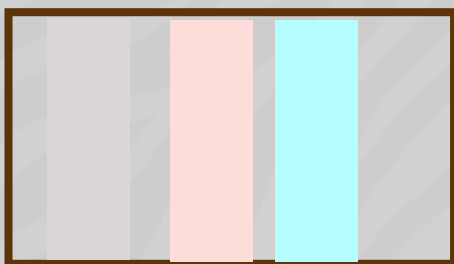
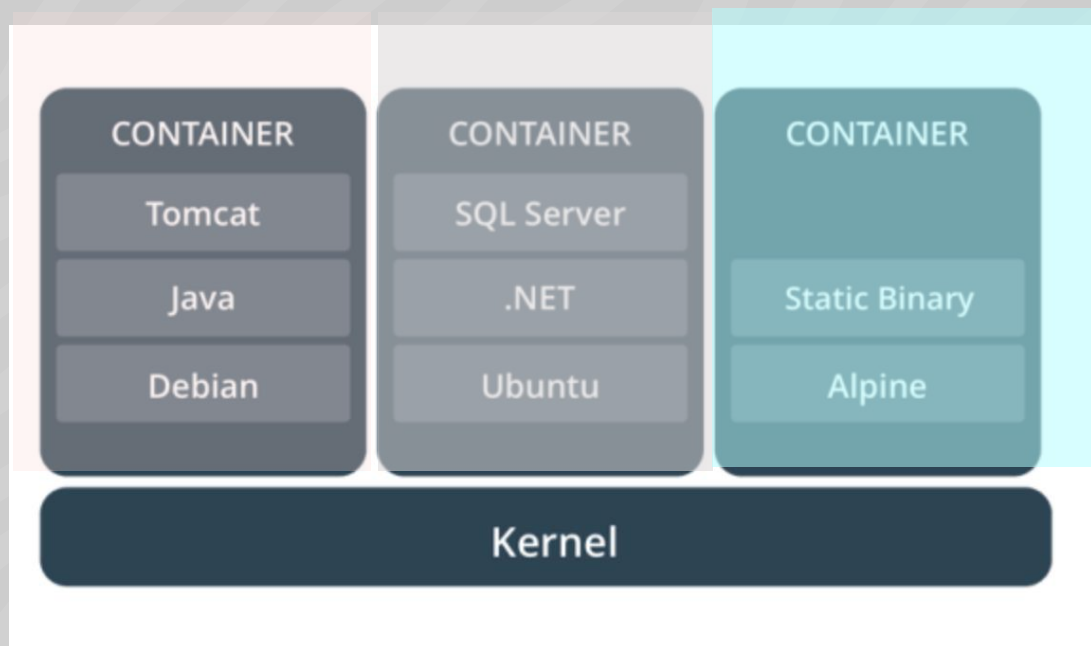
<https://www.excelproject.org/event/content>

'26 eScience





Containers provide constrained resource isolation



CPU



Memory



Filesystem



Network



Authors must program a Dockerfile

```

1  # fetch node v4 LTS codename argon
2  FROM node:argon
3
4  # Request samplename build argument
5  ARG samplename
6
7  # Create app directory
8  RUN mkdir -p /usr/src/spfx-samples
9  WORKDIR /usr/src/spfx-samples
10
11 #Install app dependencies
12 RUN git clone https://github.com/SharePoint/sp-dev-fx-webparts.git .
13 WORKDIR /usr/src/spfx-samples/samples/$samplename
14
15 # install gulp on a global scope
16 RUN npm install gulp -g
17
18 # RUN ["npm", "install", "gulp"]
19 RUN npm install
20 RUN npm cache clean
21
22 # Expose required ports
23 EXPOSE 4321 35729 5432
24
25 # Run sample
26 CMD ["gulp", "serve"]
27

```

```

FROM buildkit-export AS buildkit-buildkitd.oci_only
COPY --from=buildkitd.oci_only /usr/bin/buildkitd.oci_only /usr/bin/
COPY --from=buildctl /usr/bin/buildctl /usr/bin/
ENTRYPOINT ["buildkitd.oci_only"]

# Copy together all binaries for containerd worker mode
FROM buildkit-export AS buildkit-buildkitd.containerd_only
COPY --from=runc /usr/bin/runc /usr/bin/
COPY --from=buildkitd.containerd_only /usr/bin/buildkitd.containerd_only /usr/bin/
COPY --from=buildctl /usr/bin/buildctl /usr/bin/
ENTRYPOINT ["buildkitd.containerd_only"]

FROM alpine AS containerd-runtime
COPY --from=runc /usr/bin/runc /usr/bin/
COPY --from=containerd /go/src/github.com/containerd/containerd/bin/containerd* /usr/bin/
COPY --from=containerd /go/src/github.com/containerd/containerd/bin/ctr /usr/bin/
VOLUME /var/lib/containerd
VOLUME /run/containerd
ENTRYPOINT ["containerd"]

FROM buildkit-${BUILDKIT_TARGET}

```



Containers do not reduce technical debt

- Declarative encapsulation of dependencies for isolated execution
 - E.g. various shell utilities and library versions unknown to user



Sciunit steps and external requirements



1. Create



2. Share



3. Repeat



Mizzou®

'26 eScience

